

Highlights

Minimal Data, Maximum Clarity: A Heuristic for Explaining Optimization

Amirali Rayegan, Tim Menzies

- **Maximum clarity heuristic:** better optimization with less but key data
- **Introducing EZR:** an interpretable and modular tool for multi-objective optimization
- **Comprehensive validation:** On 60 datasets for robust optimization results
- **Explanation:** Practical, actionable explanations to guide real-world decisions
- **Downstream validation:** showing explanations improve feature selection

Minimal Data, Maximum Clarity: A Heuristic for Explaining Optimization

Amirali Rayegan^a, Tim Menzies^a

*^aDepartment of Computer Science, College of Engineering, North Carolina State
University, 890 Oval Drive, Raleigh, 27695, North Carolina, USA*

Abstract

Efficient, interpretable optimization is a critical but underexplored challenge in software engineering, where practitioners routinely face vast configuration spaces and costly, error-prone labeling processes. This paper introduces EZR, a novel and modular framework for multi-objective optimization that unifies active sampling, learning, and explanation within a single, lightweight pipeline. Departing from conventional wisdom, our Maximum Clarity Heuristic demonstrates that using less (but more informative) data can yield optimization models that are both effective and deeply understandable. EZR employs an active learning strategy based on Naive Bayes sampling to efficiently identify high-quality configurations with a fraction of the labels required by fully supervised approaches. It then distills optimization logic into concise decision trees, offering transparent, actionable explanations for both global and local decision-making. Extensive experiments across 60 real-world datasets establish that EZR reliably achieves over 90% of the best-known optimization performance in most cases, while providing clear, cohort-based rationales that surpass standard attribution-based explainable AI (XAI) methods (LIME, SHAP, BreakDown) in clarity and utility. These results endorse “less but better”; it is both possible and often preferable to use fewer (but more informative) examples to generate label-efficient optimization and explanations in software systems. To support transparency and reproducibility, all code and experimental materials are publicly available at <https://github.com/amiiralii/Minimal-Data-Maximum-Clarity>.

Keywords: Multi-objective optimization, explainable AI, Active Learning, Naive Bayes, Regression, Feature Selection, EZR

1. Introduction

This paper addresses the challenge of multi-objective optimization in software engineering (SE), such as configuration tuning and hyperparameter optimization. Selecting optimal configurations is critical for system performance, reliability, and cost-efficiency. However, in practice, optimization is often poorly managed. Software systems expose numerous configuration options, yet practitioners typically rely on defaults or intuition, leading to sub-optimal results and frequent failures [1]. Many systems are deployed with generic default configurations that are rarely revisited, even though they may not suit the target environment. Administrators often retain these defaults due to limited time, expertise, or tool support, which can lock systems into suboptimal performance states [2]. This gap between potential and practice means that large parts of the configuration space remain unexplored, leaving valuable performance gains untapped and making systems more prone to inefficiencies and hidden defects. In fact, empirical data shows that over 60% of configuration variables remain unchanged after their initial creation, highlighting how rarely these options are actively optimized [3]. This becomes a major pain point because exhaustive evaluation is infeasible, given the vast configuration spaces and the complex, non-linear interactions among parameters [4].

A major bottleneck in this task is the *labeling problem*. While collecting unlabeled configurations is easy, obtaining their labels is expensive, time-consuming, and often noisy. Creating labeled datasets in software engineering is costly and slow, as it requires expert knowledge to interpret complex artifacts like bug reports. For example, running performance tests for all configurations is computationally prohibitive [5]. Defect labels, derived heuristically from commit messages, are error-prone [6, 7]. Studies show that even carefully labeled datasets often contain errors, and the high cost of manual labeling limits the scale of available data [8]. Therefore, SE optimization methods must:

- Minimize labeling cost, using as few labels as possible.
- Identify near-optimal configurations efficiently without full Pareto front¹ estimation.

¹The Pareto front is the set of non-dominated solutions in a multi-objective problem, representing trade-offs where improving one objective requires sacrificing another. Solutions on the Pareto front are all considered optimal.

- While providing interpretable outputs, enabling practitioners to understand recommendations.

To address these goals, we propose a counterintuitive yet effective heuristic:

The Maximum Clarity Heuristic:

To explain complex tasks, **use less data**.

Consider decision tree learning. Trained on tens of thousands of examples, such trees often span many pages, making them difficult to read and even harder to reason about. But suppose we can identify just 32 highly representative exemplars from the same data. Training the same decision tree learner on that subset (with a minimum leaf size of 2) yields a tree of no more than 16 lines, referencing at most 8 variables (given binary splits). This concise structure can often communicate core decision logic far more effectively than a sprawling tree trained on the full dataset. Figure 1 shows the result of our trials toward this goal. It splits the whole data space into handful representative leaves. The column *n* shows the number of data samples on each branch and the column *win* shows the winning rate of each branch, stating how good that branch is, based on our goal and our metric which is described thoroughly in the next sections. Lines with the ; Symbol at the end of them are the leaves of the tree.

win	n	
15	32	
45	22	feature_1 <= 5:
20	9	feature_2 <= 4:
5	5	feature_3 = 'a' ; (leaf)
30	4	feature_3 != 'a' ; (leaf)
80	13	feature_2 > 4:
75	7	feature_4 <= 100 ; (leaf)
86	6	feature_4 > 100 ; (leaf)
-10	10	feature_1 > 5:
-5	5	feature_4 <= 50 ; (leaf)
-20	5	feature_4 <= 50 ; (leaf)

Figure 1: EZR generates a very small tree summarizing the major points of the data. Leaf labels denote cohorts. For a larger example, see Figure 6

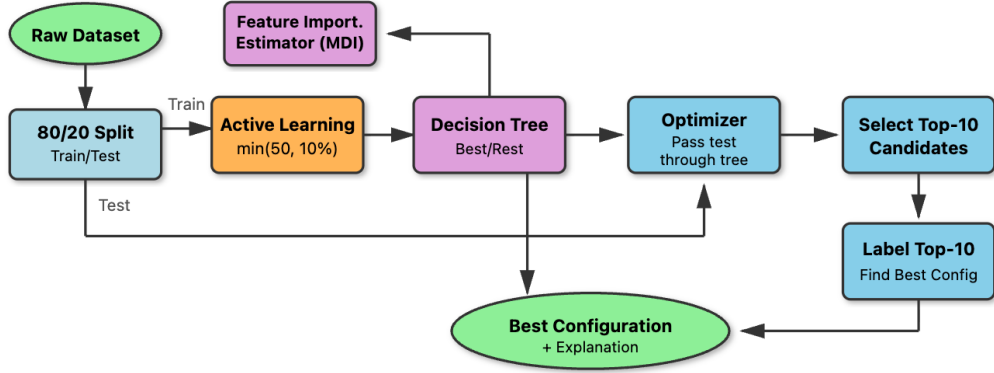


Figure 2: EZR pipeline

We introduce EZR, a novel and modular framework that unifies optimization, learning, and explanation into a single pipeline. As Figure 2 shows, EZR’s source code integrates the following core modules:

- *Active Learning Sampler*: selects the most informative data points to minimize labeling costs.
- *Decision Tree Generator*: builds interpretable models that guide structure optimization.
- *Feature Importance Estimator*: quantifies global contributions of features.
- *Optimizer*: estimates the likelihood that unlabeled data points achieve optimal multi-objective trade-offs.
- *Result Explainer*: provides local explanations of why a data point is chosen as the reference optimal and which features drive the decision.

While EZR is inspired by reinforcement learning [9], active querying, and Pareto optimization [10], it differs in key ways. Rather than relying on trial-and-error with reward signals, it employs a Naive Bayes sampling strategy that minimizes labeling effort and remains efficient. Unlike traditional active querying, EZR integrates sampling with an interpretable model structure, so each query aids both optimization and explanation. And instead of focusing only on Pareto frontiers, EZR embeds this objective within a modular framework that also supports prediction, feature ranking, and explanation. These distinctions position EZR as a lightweight yet practical alternative for software engineering tasks.

1.1. Problem Statement and Research Questions

In this work, we address multi-objective optimization problems in search-based software engineering where each candidate configuration is represented by a feature vector $f = (f_1, f_2, \dots, f_n)$ (e.g., configuration options, process parameters). Evaluating a configuration f in the target environment yields multiple objective values (e.g., performance, cost, reliability), but each such evaluation is expensive and often noisy, so only a small labeling budget is available. Our first goal is therefore to design a method that can find high-quality trade-off solutions under a tight label budget, remaining competitive with state-of-the-art optimizers that use many more labeled examples (RQ1). Our second goal is to provide clear, interpretable explanations of these solutions, so that practitioners can understand which features matter most, how changes to them affect the objectives, and why particular configurations are recommended (RQ2).

Even if we achieve label-efficient optimization and good explanations, a further bottleneck remains. There is no widely accepted, practical way to compare different explanation (XAI) methods in this setting. Existing approaches typically rely on proxy metrics or user studies, which are hard to apply systematically across many software engineering optimization tasks. To address this, we propose to compare XAI methods via their impact on downstream optimization performance, using their feature rankings as the basis for feature selection in a separate optimizer, and this motivates our third research question (RQ3). In conclusion, we formulated the following research questions:

- **RQ1: Effectiveness on optimization.** To what extent can EZR discover near-optimal configuration settings, and how does its performance compare to state-of-the-art models? *We show that EZR consistently achieves near-optimal performance ($\geq 90\%$ of best-known results in 73% of datasets) while using only a fraction of the labels required by fully supervised models.*
- **RQ2: Comparison to standard XAI.** To what extent can EZR’s “less is more” heuristic generate explanations whose clarity and actionable insight match or exceed those produced by established XAI techniques? *We show that EZR delivers clearer and more actionable explanations than attribution-based methods, spanning all rungs of Pearl’s Ladder of Causation while relying on fewer, more informative features.*

- **RQ3: Practical utility of explanations.** To what extent can EZR’s explanations generate feature rankings that improve downstream optimization performance, and how does this compare to other feature ranking methods? *We show that EZR’s explanation-driven feature rankings improve downstream optimization, performing on par with or better than established feature ranking methods while requiring far fewer labels.*

All code and experimental materials for this research are publicly available at <https://github.com/amiiralii/Minimal-Data-Maximum-Clarity>, enabling full reproducibility of the reported results.

The rest of this paper is organized as follows. Section 2 reviews related work and motivates our “less but better” approach with a real-world case study. Section 3 details the design and implementation of EZR, describes our experimental setup, and summarizes the datasets and evaluation metrics. Section 4 presents a comprehensive evaluation of EZR’s optimization, feature-selection, and explanation capabilities. Section 5 interprets these results, discusses practical implications and limitations, and lastly, Section 6 concludes with a summary of contributions and directions for future work.

2. Background

2.1. Multi-objective Optimization and Configuration Tuning in SE

Many problems in software engineering involve balancing competing objectives, such as maximizing performance while minimizing cost, or improving reliability without increasing resource usage. These trade-offs are naturally addressed through multi-objective optimization techniques, which aim to approximate the Pareto front space. In software engineering, multi-objective optimization has been applied to diverse tasks, including software configuration tuning [3, 4], test case prioritization [11, 12], requirements selection [13, 14], and project scheduling [15].

For such multi-objective problems, Search-Based Software Engineering (SBSE) has emerged as a powerful approach for systematically exploring large solution spaces and balancing conflicting objectives in modern software systems [16]. SBSE has popularized the use of metaheuristic algorithms (e.g. NSGA-II) for exploring high-dimensional and non-linear optimization spaces [15, 17]. However, these approaches often require large numbers of evaluations, which can be costly in SE contexts where each evaluation may involve

building, deploying, and testing complex systems. For example, Nair et al. [18] show in the FLASH framework that configuration optimization is often constrained by the high cost of obtaining labeled performance data, and that label-efficient methods can dramatically reduce the number of evaluations needed.

Evaluating candidate solutions in Pareto-based SBSE is a non-trivial task [19]:

- Recent results at ICSE’26 cast doubts on the value of reasoning over the whole Pareto frontier. The PROMISE-TUNE [20] study reports that the best solutions are typically clustered into a very small region of the total Pareto frontier. This means that traditional assessment methods (hypervolume, spread, GD, IGD) are highly misleading since those measures comment on the whole frontier (and not the tiny regions containing the best solutions).
- While traditional multi-objective optimization methods aim to approximate the entire Pareto front, such approaches require substantial computational resources and large numbers of labeled evaluations to build accurate surrogate models of the search space [18, 19].
 - For instance, methods like NSGA-II and MOEA/D rely on dominance relationships that necessitate pairwise comparisons across the population, while indicator-based approaches require either prior knowledge of the true Pareto front or expensive approximations thereof [21].
 - Also, another reason not to use standard Pareto-based metrics is that recent work has demonstrated that computing quality indicators for multi-objective solutions often demands extensive labeled data to establish reference points or ideal vectors [22–24].
- In contrast, our goal is fundamentally different. Inspired by the PROMISE-TUNE results [20], rather than modeling the entire solution space or estimating the complete Pareto front, EZR seeks to efficiently identify a small number of high-quality configurations under severe label constraints. This motivates the need for a ranking mechanism that is computationally lightweight (requiring no iterative population updates or dominance sorting) and label-efficient, which operates without access to the full Pareto front.

Moreover, recent work has shown that the search spaces of SE problems often contain redundant or weakly relevant dimensions [25], meaning that exhaustive search wastes effort on uninformative configurations. This motivates the need for approaches that can intelligently allocate budget toward the most promising regions of the search space.

While multi-objective optimization techniques have achieved notable successes in SE, their practical deployment is limited by evaluation cost, noisy objectives, and the difficulty of interpreting why certain solutions are preferred. This creates an opportunity for lightweight, explanation-oriented optimization methods (such as EZR) that can produce competitive results using fewer evaluations while also providing interpretable decision rationales.

To illustrate a typical optimization scenario in software engineering, consider the **COC1000** dataset from the MOOT repository². This dataset models a *COCOMO-style* software process, where the objective is to optimize project planning by minimizing several undesirable outcomes. Each row in the dataset represents a hypothetical project configuration described by 20 independent variables, all described in Table 1. The dependent goals include:

- **Risk-**: Project risk (to be minimized)
- **Effort-**: Estimated person-months (to be minimized)
- **AEXP-**: Required analyst experience (to be minimized)
- **PLEx-**: Programming language experience required (to be minimized)
- **LOC+**: Delivered lines of code (to be maximized)

The optimization task is to identify configurations that *minimize risks and resource needs while maximizing productivity*. Such trade-offs are typical in software project management, where cost, staffing demands, and output must be balanced. In our experimental setup, the true values of the objectives remain hidden until explicitly queried, and optimization performance is assessed by the actual quality of the selected configurations while keeping the number of queries (i.e., labels) as low as possible. The dataset contains **1,001 configurations**, which makes exhaustive evaluation impractical. Therefore, optimizers must select rows *intelligently*, incurring a cost for each query (i.e., when the true goal values are revealed). Note that the 20 independent variables in this dataset represent factors that can take managers years to learn to balance effectively [26].

While efficient optimization is essential, it is not sufficient on its own.

²<https://github.com/timm/moot/blob/master/optimize/process/coc1000.csv>

Feature	Description
ACAP	Analyst capability: skill level and experience of analysts
PCAP	Programmer capability: general skill of software developers
CPLx	Product complexity: structural and algorithmic complexity
DATA	Database size: relative size of data in the application
DOCU	Documentation match: completeness and match to lifecycle needs
PMAT	Process maturity: software process capability
RELY	Required reliability: criticality and dependability requirements
SCED	Schedule constraint: compression or expansion of timeline
SITE	Team distribution: communication overhead across sites
STOR	Storage: main memory constraints of the software product
TIME	Execution time constraints: Performance requirements relative to available computing power
TOOL	Tool support: availability and integration of development tools
ARCH	Architecture and Risk Resolution: Quality of architecture and risk resolution
FLEx	Development Flexibility: Degree of flexibility in development processes
LTEx	Language and Tool Experience: Familiarity with programming languages and tools
PCON	Personnel Continuity: Turnover rate or stability of the project team
PREC	Precedentedness: Familiarity with the project type
PVOL	Platform Volatility: Frequency of hardware/software changes
RUSE	Required Reusability: Level of reusability required for components
TEAM	Team Cohesion: How well the team works together

Table 1: Independent variables in the *coc1000* dataset

In practice, optimizers must also justify their choices so that practitioners can trust and act on them, making explainability a critical complement to optimization. This brings us to the role of explainable AI in software engineering.

2.2. Explainable AI in SE

AI and machine learning models are increasingly used in software engineering for tasks such as defect prediction, code recommendation, automated testing, and maintenance. Although these models often achieve strong predictive performance, their complexity produces “black-box” solutions whose inner workings are opaque to practitioners and decision makers. This lack of interpretability poses both practical and ethical challenges. Stakeholders hesitate to rely on AI tools they cannot understand or justify, especially in critical activities like bug fixing, code review, or requirements analysis. As a result, state-of-the-art models are often viewed as unreliable, costly to validate, or risky to deploy, leading to reduced trust, excessive manual oversight, or outright rejection [27]. Crucially, many SE tasks demand not only predictions but also actionable explanations. For example, in defect prediction, simply flagging a file as defective is insufficient and developers and managers need to know why the model reached that conclusion to debug, validate, and align decisions with project goals [28]. This demand underscores the importance of explanations that are both accurate and actionable. Stakeholders frequently ask questions such as “Why is this file predicted as defective?” or “Why has this task been recommended for maximum development effort?” Addressing such concerns requires explanations that practitioners across roles, developers, managers, analysts, legal teams, and executives, can trust, validate, and act upon [29, 30]. Recent SE research has therefore shifted from coarse, opaque predictions toward models that provide clear and practical guidance. Explanations should be delivered in accessible forms, such as natural language, decision trees, visual highlights, or actionable recommendations, tailored to the needs of different stakeholders [29]. To support this, traditional black-box models are increasingly complemented by interpretability techniques. White-box methods expose feature importance in deep learning models, while black-box methods rely on perturbation and attribution strategies to provide model-agnostic explanations at both local and global levels. Importantly, these explanations must remain understandable to non-experts, avoiding overly technical presentations that hinder adoption [31].

Explanations in AI are generally distinguished as global or local. *Global explanations* describe overall model behavior, identifying which features most influence predictions across the dataset. In this study, we employ the permutation importance framework [32], a model-agnostic technique that quantifies each feature’s contribution by permuting its values and measuring the resulting drop in model accuracy. *Local explanations*, in contrast, clarify why a model produced a particular outcome for an individual instance. To evaluate these, we compare EZR against three widely used methods. LIME (Local Interpretable Model-agnostic Explanations) [33] builds a sparse linear surrogate around the target instance to highlight the most influential features locally. SHAP (SHapley Additive exPlanations) [34] applies cooperative game theory to compute fair, additive attributions that sum to the difference between the model’s prediction and a baseline. BreakDown [35] sequentially decomposes a prediction by adding features one at a time in a fixed order, producing a step-by-step narrative of how each feature shifts the output away from the baseline.

The justification for incorporating explanations into AI systems, as Chen et al. [36] argue, arises from a fundamental limitation of purely data-driven optimization: For an analytic model to be adopted and create value, it must persuade users of its findings. This necessity follows from two core psychological principles. First, humans have limited cognitive capacity, as noted by Miller’s “magical number seven” [37]. Stakeholders cannot reason about hundreds of variables simultaneously. Without simplified narratives, results from high-dimensional models remain cognitively inaccessible. Explanations reduce this complexity, distilling decisions into understandable rules or factors aligned with human reasoning. Second, stakeholders often hold pre-existing beliefs and intuitions about their domain, and they often experience cognitive dissonance when model outputs contradict their prior beliefs [38]. Without a compelling rationale, they are more likely to reject the model than update their mental model. Explanations act as persuasive tools, resolving dissonance and enabling acceptance of counterintuitive results.

Beyond these psychological foundations, the rise of opaque “black-box” models has intensified the need for explanation. Three additional arguments reinforce its importance:

1. **Fostering Trust in High-Stakes Decisions:** Explanations transform blind reliance into informed confidence, a necessity in domains where errors carry high costs.

2. **Ensuring Ethical and Regulatory Compliance:** Explanations enable audits for hidden bias and support legal frameworks such as the EU’s “right to explanation” under GDPR [39]. Transparency has thus become a core system requirement [40].
3. **Supporting Debugging and Discovery:** For developers and scientists, explanations are indispensable for diagnosing unexpected model behavior and for revealing novel patterns that drive new hypotheses.

In sum, explanations have evolved from a psychological aid into a multifaceted requirement. They build trust, ensure fairness and accountability, and advance scientific understanding. Far from being an optional feature, explanation is now a foundational pillar of modern, responsible AI.

3. Methods

3.1. Proposed Method: EZR

This study introduces EZR, a new multi-purpose framework designed to efficiently perform machine learning tasks in software engineering domain.

By combining active learning’s label efficiency with decision trees’ inherent interpretability, EZR achieves what Menzies [41] characterizes as "shockingly simple" solutions that almost match or exceed the performance of far more complex alternatives. Recent work on hyperparameter optimization has emphasized the importance of matching algorithmic complexity to problem complexity. Agrawal et al. [42] proposed using intrinsic dimensionality thresholds ($I < 5$) to select between simple and complex optimizers. That is, when the problem is intrinsically simple, then simple optimizers will suffice. EZR operationalizes this principle by using minimal labeled data to construct interpretable models that rival fully supervised approaches. This stands in contrast to methods like DEHB [43], which require thousands of evaluations, or traditional Bayesian optimization with Gaussian Process Models that scale poorly with dataset size [18].

EZR builds upon a rich lineage of hyperparameter optimization and explainable AI research, but it departs from its predecessors in fundamental ways. While inspired by the Tree of Parzen Estimators (TPE) framework proposed by Bergstra and Bengio [44], EZR simplifies the surrogate modeling approach considerably:

- TPE constructs full kernel density estimators to model the probability distributions of "good" and "bad" configurations, requiring multiple kernels to estimate both mean and variance of predictions.

- In contrast, EZR replaces this complex density estimation with a lightweight Naive Bayes classifier that directly computes likelihood ratios between "best" and "rest" configurations.

This architectural choice aligns with recent findings by Lustosa [25, 45], who demonstrated that many software engineering optimization problems exhibit surprisingly low intrinsic dimensionality, often reducing datasets with dozens of raw attributes to just a handful of intrinsic dimensions.

EZR leverages this insight from Lustosa by employing a simpler acquisition strategy that avoids the exploration-exploitation balancing acts typical of reinforcement learning approaches [9]. Rather than dynamically adjusting between exploratory and exploitative phases, EZR uses a greedy elite-search that focuses sampling budget on configurations with the highest likelihood ratios, effectively performing a Hoare-style quickselect [46] over the configuration space.

EZR’s design philosophy also extends to its treatment of interpretability. Where established explainable AI methods like LIME, SHAP, and Break-Down provide post-hoc, attribution-based explanations that remain confined to Pearl’s associational rung of causation [47], EZR’s decision tree structure natively supports all three rungs: associations (through global feature importance), interventions (through threshold-based what-if analysis), and counterfactuals (through alternative leaf assignments).

Algorithm 1 EZR Overall Pipeline

Require: dataset name s

```

1:  $raw \leftarrow \text{DATA}(\text{CSV}(s))$ 
2:  $train, test \leftarrow \text{SPLIT}(raw, 0.8)$ 
3:  $budget \leftarrow \min(50, |train|/10)$ 
4:  $acq \leftarrow \text{XPLOIT}$ 
5:  $k \leftarrow 10$ 
6:  $Labels \leftarrow \text{ACTIVELEARN}(train, acq, budget)$ 
7:  $Root \leftarrow \text{TREE}(Labels)$ 
8:  $scores \leftarrow \text{PASSTOTREE}(Root, test)$ 
9:  $C \leftarrow \text{TOPK}(scores, k)$ 
10:  $bestRow, performance \leftarrow \text{WIN}(\min_{r \in C} \text{D2H}(\text{LABEL}(r)))$ 
11: return ( $bestRow, performance$ )

```

To provide an overview of the complete workflow, Figure 2 and Algorithm 1 outline the main stages of EZR. The process begins by splitting

the dataset into training and testing subsets (80/20). An active learning stage then selects $\min(50, 10\% \text{ of the training})$ samples for labeling using the *xploit* acquisition strategy. The labeled subset is then used to train an interpretable decision tree. Next, each test instance is subsequently passed through the tree to estimate its utility based on the statistical summary of the leaf it falls into. Finally, the top 10 candidates with the highest estimated performance are labeled to identify the true best configuration. This way the final labeling budget would be 60 (or 10% of the training + 10). The following sections provide detailed explanations of the components outlined in Algorithm 1, including active learning, decision tree generation, evaluation methods and win-rate calculation, and budget assignment.

3.1.1. Ranking Configurations

A critical design decision in EZR is using a single, unified metric to rank configurations for multi-objective optimization. EZR uses distance-to-heaven (d2h), computed as the Euclidean distance between each row’s normalized objective values and an ideal “heaven” point with optimal values for each objective (since objectives are normalized, each heaven coordinate is 0 or 1, depending on whether that objective is minimized or maximized). This choice meets our two needs: it is computationally lightweight and label-efficient, and it avoids Pareto-front estimation. Configurations with low d2h are guaranteed to be near-Pareto-optimal, since any dominated solution must have a larger distance to the ideal point. Equation 1 defines this single-scalar ranking per row, where m is the set of objectives and o_i is the normalized value of objective i .

$$\text{d2h}(x) = \sqrt{\sum_{i=1}^m (o_i(x) - h_i)^2} \quad \text{where} \quad h_i = \begin{cases} 1 & \text{Maximizing } i \\ 0 & \text{Minimizing } i \end{cases} \quad (1)$$

3.1.2. Active Learning

EZR frames configuration optimization as an active learning problem. It begins with a small set of randomly labeled configurations to bootstrap an initial model. Then, at each iteration, it trains a two-class Naive Bayes classifier to distinguish between "Best" and "rest" configurations. Here, "Best" refers to the top-performing $\sqrt{N}$ configurations based on their d2h values,

while "rest" includes the remaining $N - \sqrt{N}$ configurations. For each unlabeled configuration, EZR calculates the likelihood of it belonging to the "Best" class divided by the likelihood of it belonging to the "rest" class. The configuration with the highest such ratio is selected for labeling next. This process repeats until a predefined labeling budget is exhausted. By prioritizing configurations that are most likely to improve optimization outcomes, EZR minimizes the number of required labels.

Algorithm 2 Active Learning with XPLOIT

```

1: function LIKE( $x$ , model)
2:   return Naive-Bayes log-likelihood with prior and likelihoods
3: end function
4: function ACTIVELEARN( $D$ )
5:    $n \leftarrow 4$  ▷ Warm Start
6:    $stop \leftarrow 50$  ▷ Predefined Labeling Budget
7:   shuffle  $D.rows$ 
8:    $todo \leftarrow D.rows[n:]$ 
9:    $done \leftarrow D.rows[:n]$ 
10:   $cut \leftarrow \text{round}(n^{0.5})$ 
11:   $best \leftarrow done[:cut]$ 
12:   $rest \leftarrow done[cut:]$ 
13:  while  $n < stop$  do
14:     $n \leftarrow n + 1$ 
15:     $cut \leftarrow \text{round}(n^{0.5})$ 
16:     $h \leftarrow \max_{x \in todo} \frac{Like(x, best)}{Like(x, rest)}$  ▷ XPLOIT: ratio  $B/R$ 
17:     $best \leftarrow best \cup \{h\}$ 
18:    if  $|best.rows| \geq cut$  then
19:      Move weakest in best  $\rightarrow rest$ 
20:      Pop weakest in rest
21:    end if
22:  end while
23:  return ( $best, rest, todo$ )
24: end function

```

3.1.3. Parzen Tree

EZR is inspired by TPE, which models good and bad configurations using Parzen estimators (kernel density estimators) to guide sampling decisions. However, instead of constructing full density estimators, EZR simplifies this process by using a Naive Bayes classifier to approximate the likelihood of configurations being "best" versus "rest". After the active learning loop completes, EZR builds a decision tree over the labeled configurations to extract interpretable rules explaining which features drive optimal outcomes. The tree function partitions configurations based on feature splits that minimize the variance of $d2h$ within each branch, producing clear and human-readable explanations. This combination of lightweight active learning and post-hoc

decision tree analysis allows EZR to efficiently find high-performing configurations with minimal labels while providing insights into the underlying decision rationale.

In addition to its optimization capability, EZR also functions as a feature-importance ranking tool by leveraging its interpretable decision tree. At each internal node, EZR evaluates how well a feature partitions the data by measuring the reduction in $d2h$ variance, recorded as the node’s *impurity*. The overall importance of a feature is then obtained by summing its impurity reductions across the tree, directly aligning with the Mean Decrease in Impurity (MDI) framework used in tree-based models such as Random Forests [48]. Formally, the importance of feature f is defined as:

$$MDI(f) = \sum_{n \in \mathcal{N}_f} \sum_{c \in C(n)} \frac{|c|}{\sum_{j \in C(n)} |j|} \cdot Imp(c) \quad (2)$$

where $\mathcal{N}_f$ is the set of internal nodes where f is used for splitting, $C(n)$ denotes the children of node n , and $Imp(c)$ is the standard deviation of $d2h$ values within child node c . This formulation highlights features that meaningfully reduce performance variability within configuration clusters.

3.1.4. Compared to other methods

While certainly inspired by the literature on reinforcement learning and active learning, EZR is far simpler. For example: instead of applying multiple kernels to learn the mean and variance of any prediction (as in Hyperopt [10]), EZR just uses a simple nearest-neighbor check against the centers of best and rest. Also, instead of balancing exploration and exploitation (as done in reinforcement learning [9]) EZR just uses the above greedy elite-search.

4. Experimental setup

In this study, we conducted an extensive set of experiments to evaluate the effectiveness of our proposed method. The experimental design was structured to systematically assess EZR’s performance in optimization and explainability compared to widely used state-of-the-art methods in software engineering.

4.1. Optimization Performance: Regression-Based Baselines

The goal of our experiment is to evaluate the effectiveness of EZR as an optimizer. Before comparing EZR directly to traditional optimization

algorithms, we benchmark it against regression-based predictors. For each dataset, several regression models were trained to predict the $d2h$ values of configurations. Each model then recommended the configuration with the lowest predicted $d2h$, which we evaluated by comparing its true $d2h$ value to that of EZR’s recommendation. This allows us to assess how well each approach identifies configurations near the ideal multi-objective trade-offs. Practically, all of our datasets include ground-truth labels (e.g., true risk, cost, or duration), and thus all $d2h$ values are available. However, in our experiment we assume these labels are hidden during optimization, revealing them only at the end to validate the quality of the selected configurations.

To ensure a competitive and comprehensive comparison, we utilized five widely used regression methods popular in the software engineering domain. Each method represents a distinct family of regression algorithms, enabling us to benchmark EZR against a diverse set of predictive approaches. Linear Regression(LR) was chosen to represent classic statistical models, while Random Forest(RF) serves as a representative of tree-based ensemble methods. Light Gradient Boosting Machine (LightGBM or LGBM) captures the performance of boosting algorithms, and a simple Artificial Neural Network(ANN) was included to represent deep learning-based regressors. Additionally, Support Vector Regressor(SVR) was selected to represent kernel-based methods that are effective in capturing complex non-linear relationships. As a baseline, we also included Random Selection, which randomly selects a configuration as the optimal setting, to contextualize the performance gains achieved by other methods. In total, this setup yields seven treatments per dataset; EZR, the five regression models, and the random baseline.

4.2. Optimization Performance: Optimization Baselines

To position EZR within the broader optimization literature, we compare it against two widely used state-of-the-art optimizers, SMAC (Sequential Model-based Algorithm Configuration) [49, 50] and DEHB (Distributed Evolutionary Hyperband) [43], alongside a random baseline. Both SMAC and DEHB are popular in software engineering and automated machine learning and represent state-of-the-art membership query inference methods for hyperparameter optimization. DEHB combines differential evolution search with Hyperband’s adaptive resource allocation, using successive halving to evaluate configurations, and is considered the current state-of-the-art, outperforming BOHB and Hyperopt [51]. SMAC utilizes Random Forests to model the objective function, enabling support for categorical parameters,

and proposes new solutions by maximizing the Expected Improvement acquisition function. We conducted two controlled experiments with fixed evaluation budgets:

- Using 50 labels
- Using 200 labels

In each setting, all methods (EZR, SMAC, DEHB, and random) were restricted to the same number of objective evaluations to ensure fairness. Experiments were run on all 60 datasets. To reduce randomness effects, each experiment was repeated 20 times per dataset using different random seeds. In each repetition, after exhausting the budget, each optimizer selected the configuration it predicted to be best. We then revealed its true objective values and computed its *d2h* and corresponding *win* score (to be discussed in later sections). Finally, we applied statistical tests to the 20-run distributions to determine whether performance differences between methods were statistically significant.

4.3. Explainability Performance

While most prior work in software engineering has evaluated the effectiveness of XAI methods through subjective assessments (which we also did in the section 5.2.1), such evaluations, though valuable, may overlook more objective measures of explanatory utility [27]. To address this limitation, we adopt a complementary approach inspired by recent XAI evaluations in other domains [52–54]. To evaluate the effectiveness of EZR as an explanation method, we operationalize the effectiveness of an explanation method by examining whether its feature-importance rankings can improve downstream predictive performance. The intuition is that a meaningful explanation should identify features that are not only interpretable but also predictive of outcomes in downstream optimization tasks. Thus, by comparing feature subsets derived from different explanation or selection methods, we can objectively assess their utility. EZR was benchmarked against three widely used feature ranking techniques in software engineering domain, ReliefF, SHAP, and ANOVA. **ReliefF** [55] is an instance-based method that estimates feature importance by comparing how well features distinguish between neighboring instances of different classes, making it effective for handling feature interactions and multi-class problems. **SHAP** (SHapley

Additive exPlanations) [34] computes feature contributions based on Shapley values from cooperative game theory, offering theoretically grounded and consistent importance scores. **ANOVA** (Analysis of Variance) [56] evaluates the statistical significance of differences in feature means across output categories, serving as a simple yet effective filter-based method for dimensionality reduction. As a reference, we also include a **no-selection** baseline that uses all available features to assess the added value of applying feature selection. For each dataset, EZR is first applied as a feature selector to determine the number of features, k , to retain. Using this same k , all other methods are applied to select their respective top- k features. Each resulting feature subset is then used to train the LGBM regression model, and predictive performance is compared across treatments.

4.4. Evaluation Protocol

While statistical rank-based methods such as Scott-Knott [57], as used in [21, 58], offer a way to group treatments by significance, they do not convey how close or far apart the performances of different methods truly are. In our context, where models operate under strict label constraints, it is expected that no method, including EZR, can fully recover all latent information in the data. However, the value lies in how well EZR balances this limitation against labeling cost. To better capture the effectiveness of this trade-off, we introduce a relative performance metric that quantifies how close EZR gets to the best-known solution found by the strongest state-of-the-art method with access to all labels. Specifically, we define a ratio-based metric, inspired by the concept of normalized regret in reinforcement learning (equation 4), to measure EZR’s efficiency relative to fully supervised baselines.

$$\text{Relative Score (\%)} = 100 \times \frac{\text{win}(d2h_{\text{EZR}})}{\text{win}(d2h_{\text{SOTA}})} \quad (3)$$

where $\text{win}(d2h)$ denotes the normalized improvement score (higher is better). The function $\text{win}(x)$ is defined as:

$$\text{win}(x) = 100 \times \left(1 - \frac{x - \min}{\text{median} - \min} \right) \quad (4)$$

In the context of optimization, x refers to the predicted $d2h$ value of a configuration selected by a given model. The value $\min$ is the lowest (most desirable) $d2h$ observed in the dataset, and median is the median of all $d2h$

values. This function rescales outcomes such that lower $d2h$ values (closer to the ideal) receive higher win scores. A relative score close to 100% indicates that the model achieves performance comparable to the best method, despite using far fewer labels.

#Datasets	Dataset Type	File Names	Primary Objective	x/y
25	Specific Software Configurations	SS-A to SS-X, billing 10k	Optimize software system settings, Runtimes, query times, and usage data from software configured in various manners	3–88 / 2–3
1	cloud	HSMGP num	Hazardous Software, Management Program data	14/1
1	cloud	Apache AllMeasurements	Apache server performance optimization	9/1
1	cloud	SQL AllMeasurements	SQL database tuning	38/1
1	cloud	X264 AllMeasurements	Video encoding optimization	16/1
7	cloud	(rs—sol—wc)*	misc configuration tasks	6/1
2	Health Data Sets	healthClose Issues12mths	Predict project health and developer activity	5/3
3	scrum	Scrum1k, Scrum10k, Scrum100k	configurations of the scrum feature model	124/3
8	Feature Models	FFM-*, FM-*	Optimize number of variables, constraints and Clause/Constraint ratio	128–1024/3
1	software process model	COC1000	Optimize risk, effort, analyst experience, programming language experience	20/5
4	software process model	POM3 (A–D)	Balancing idle rates, completion rates and overall cost	9/3
4	flight, ground, Orbital Space Plane software	XOMO (Flight, Ground, OSP)	optimizing risk, effort, defects, and months.	27/4
2	Misc	auto93, Wine	Misc	4/3
60	Total			

Table 2: Summary of datasets used in this study.

4.5. Data

This study uses datasets from the MOOT repository (Multi-Objective Optimization Testing) [59], which compiles a diverse range of software engi-

neering optimization problems from published research³. The MOOT datasets are widely used in SE optimization studies as they cover tasks such as process tuning, configuration optimization, hyperparameter optimization (HPO), and management decision-making.

In total, we used 60 datasets categorized under five main types in MOOT:

- **Binary Configs.** This category contains datasets modeling binary configuration spaces, where each independent variable represents an on/off or enabled/disabled choice. These datasets are typically used to evaluate optimization techniques for highly configurable systems such as software product lines, where the configuration space can reach millions of possible combinations. Each dataset captures performance metrics such as runtime, memory usage, or defect counts for different feature combinations.
- **Config.** Configuration datasets focus on system parameter tuning tasks. Examples include database configurations such as MySQL or Apache server settings. Each row represents a unique configuration with measured objectives like throughput and latency. For instance, tuning buffer sizes, thread pools, or cache settings to optimize database performance is a typical problem modeled in these datasets. These tasks are practically important, as default configurations are rarely optimal and manual tuning is often infeasible due to the size of the configuration space.
- **HPO (Hyperparameter Optimization).** The HPO category includes datasets representing hyperparameter tuning tasks for machine learning algorithms. For example, optimizing the kernel type and regularization parameter for SVM, the number of estimators and maximum depth for Random Forest, or learning rate and boosting type for XGBoost. Each dataset captures the performance outcomes (e.g., accuracy, F1 score) for various hyperparameter settings, enabling comparative evaluation of optimization approaches for model tuning.
- **Process.** Process datasets capture software engineering process optimization problems, including task scheduling, effort estimation configurations, and process parameter tuning for builds or deployments. Objectives in these datasets often include minimizing total project duration, reducing cost, or minimizing defects introduced during processes.

³<https://github.com/timm/moot/tree/master/optimize>

Such problems are critical for software project management and process improvement studies.

- **Misc.** The miscellaneous category consists of diverse datasets that do not fit other categories and are not related to software engineering. These datasets increase the diversity of optimization problems tested in this study.

Across all categories, MOOT datasets are structured as tabular data with x inputs (independent variables) and y goals (dependent objectives). Numeric columns begin with uppercase letters, while symbolic columns begin with lowercase letters. Goal columns are annotated with “+” or “-” to indicate whether they are to be maximized or minimized, respectively. Dataset sizes vary from a few hundred to over 80,000 rows, with up to 88 independent variables and multiple objectives per dataset.

During experiments, rows are randomly shuffled, and y values are hidden from the optimizer until queried, simulating realistic labeling cost constraints. Each optimizer request for a label incurs a unit cost, emphasizing the practical need for minimal-label optimization approaches. This diverse dataset collection provides a comprehensive evaluation ground to assess EZR’s ability to perform feature selection, optimization, and explanation effectively across a wide range of SE tasks. Table 2 summarizes the datasets used in this study. The table reports each dataset’s number of inputs and objectives (x/y), primary optimization objective, and file name.

5. Results

5.1. How EZR is good at finding best settings?

5.1.1. Regression-Based Baselines

To better understand the performance of EZR in selecting best configurations, we present our results across three separate tables, each grouping datasets by their input feature dimensionality. This stratification helps isolate how EZR performs under different levels of complexity.

- The first table contains results for datasets with **6 or fewer features**, referred to as *light* datasets.
- The second table presents results for datasets with **between 7 and 19 features**, labeled as *medium* datasets.
- The third table includes datasets with **20 or more features**, categorized as *heavy* datasets.

Each column in the tables corresponds to a different regression model used in the optimization process. For each experimental run, the dataset is randomly split into 80% training and 20% test data. Models are trained on the training set and are tasked to identify the configuration in the test set that yields the lowest $d2h$ score. This process is repeated 20 times for each dataset, with a different random seed used in each repetition to account for sampling variability.

It is important to note that all models **except EZR** use labeled training data for learning. In contrast, **EZR operates under a label-constrained setting**, using only:

- **60** labeled configurations for *light* and *medium* datasets (labeled in two stages, first it selects 50 configurations from the pool of unlabeled settings, using active learning, and then it checks all the unlabeled data and labels the 10 best settings to find the actual best among them), and
- **10% + 10**, 10% of the training set for *heavy* datasets at the first stage and 10 more samples at the next stage.

The choice of 50 sampling budget for light and medium datasets is grounded in established theory from optimization literature. Following the geometric process framework[60] for near-optimal solution discovery, if solutions are randomly distributed and values within ε are indistinguishable, the probability of drawing an ε -optimal solution follows a geometric distribution. The number of samples needed to achieve confidence C of finding an ε -optimal solution is:

$$1 - (1 - \varepsilon)^n \geq C \quad (5)$$

which when solved for n rearranges to

$$n(c, \varepsilon) = \frac{\log(1 - C)}{\log(1 - \varepsilon)} \quad (6)$$

Cohen’s rule [61] stats that results are statistically distinguishable if they differ by more than a “small effect”. Here, “small” and “medium” are defined as ratios of a standard deviation; i.e. 0.2ε is ”small”; 0.5ε is ”medium”; and 0.35ε is the border between small and medium. From an engineering perspective, Gaussian span the range $\pm 3\varepsilon$ so to 95% confident of finding solutions indistinguishable from the best, this yields:

$$n(C = 0.95, \varepsilon = .35/6) \approx 50 \quad (7)$$

Similar theoretical reasoning has been employed by Lustosa et al. [62] in their partial-ordering explanations, drawing from Hamlet’s Probable Correctness Theory [60] and Hoare’s Quickselect pivot-and-partition strategy [46], both of which support the use of small but statistically meaningful sampling budgets in optimization and explanation tasks. In addition to these theories, to validate the theoretical budget in practice, we also conducted experiments using a range of smaller and larger budget values, and the results of this sensitivity analysis are presented in Section 6.1.

Table 3, 4, and 5 compare the performances of EZR with regression models for all datasets. The first row in each table lists the regression models used in our experiments. The column labeled *AsIs* represents a random selection baseline, which simply chooses a configuration at random. The x and y columns indicate the number of input features and optimization objectives in each dataset, respectively. The x^* column shows the number of features presents in the EZR tree branch that leads to the best leaf. In other words, in order to find the best setting with EZR only that many features are actually used. The *rows* column shows the total number of configurations in each dataset. The numbers reported in each table cell represent the **median accuracy** over 20 repeated runs, where the accuracy is measured as the $\text{win}(d2h)$ score of the best configuration identified in each run. Higher values indicate that the method consistently identified configurations closer to the ideal multi-objective trade-off. Green cells indicate the best performance achieved by any of the state-of-the-art regression models. Orange cells highlight cases where EZR finds a configuration whose performance is at least 90% as good as the best model.

As shown in Table 3, which presents the results for the *light* datasets (those with six or fewer features), EZR demonstrates competitive performance despite operating under strict label constraints. Specifically, EZR achieves at least 90% of the best-known performance in 18 out of 22 datasets, accounting for 82% of all cases. This is particularly noteworthy considering that EZR uses only 60 labeled examples per dataset, while all other regression models rely on full supervision using the entire training set. These results highlight EZR’s effectiveness in identifying near-optimal configurations with minimal labeling effort, supporting its value in practical, label-scarce SE optimization tasks.

As shown in Table 4, which reports the results for the *medium* datasets (those with 7 to 19 input features), EZR continues to exhibit strong performance under label constraints. In this setting, EZR achieves at least 90%

AsIs	LR	RF	SVR	ANN	LGBM	EZR	data	x*	x	y	budget	rows
20	93	93	93	91	93	92	SS-A	3	3	2	60	1344
-17	84	84	84	84	84	84	SS-B	2	3	2	60	207
25	80	87	83	79	87	80	SS-C	2	3	2	60	1513
17	73	85	84	54	85	72	SS-D	3	3	2	60	197
25	82	97	97	83	97	91	SS-E	3	3	2	60	757
33	76	91	91	73	91	83	SS-F	3	3	2	60	197
23	76	92	92	60	92	81	SS-G	3	3	2	60	197
-15	95	99	99	99	99	99	wc-1	2	3	1	60	197
1	95	99	99	99	99	99	wc-2	2	3	1	60	197
-2	95	99	99	98	99	99	wc-3	2	3	1	60	197
6	100	100	100	100	100	100	SS-H	3	4	2	60	260
18	98	98	42	98	98	98	SS-I	4	5	2	60	1081
36	94	94	94	78	94	94	auto93	4	5	3	60	399
-31	36	64	49	37	64	42	HCI-hard	4	5	3	60	10001
-22	100	100	100	100	100	100	HCI-easy	3	5	3	60	10001
-13	97	96	95	96	98	97	SS-J	5	6	2	60	3841
1	61	98	94	90	98	81	SS-K	5	6	2	60	2881
50	99	100	100	94	100	95	SS-S	3	6	2	60	3841
43	100	100	99	100	100	100	rs-1	4	6	1	60	3841
92	99	100	99	100	100	100	rs-2	3	6	1	60	3841
-1	93	95	94	94	94	94	sol-1	4	6	1	60	2867
11	100	100	100	100	100	100	wc-4	3	6	1	60	2881

Table 3: EZR Performance comparison for light datasets

of the best-known performance in 11 out of 19 datasets, accounting for 58% of the cases. As with the *light* datasets, EZR operates with only 60 labeled examples per dataset, while the other regression models are fully supervised using the entire training set.

As shown in Table 5, which presents the results for the *heavy* datasets (those with more than 20 input features), EZR achieves its strongest performance under label constraints. In this group, EZR reaches at least 90% of the best-known performance in 17 out of 19 datasets, amounting to 89% of all cases—its highest success rate across all dataset categories. Notably, for these larger datasets, EZR uses only $10 + 10\%$ of the available training labels.⁴ This high efficiency is particularly significant because these heavy datasets are more representative of real-world scenarios, where the configuration space is large, the number of tunable parameters is high, and labeling costs are substantial. EZR’s ability to perform well in such settings highlights its practicality for real-world software engineering optimization tasks.

⁴Except for coc1000 and scrum1k, which have a labeling budget of 60 since they have fewer rows.

AsIs	LR	RF	SVR	ANN	LGBM	EZR	data	x*	x	y	budget	rows
45	95	95	95	95	95	95	Apache-AM	6	9	1	60	193
-14	93	88	87	75	88	79	pom3a	4	9	3	60	20001
-7	88	87	87	79	87	65	pom3b	4	9	3	60	20001
-3	79	69	78	50	68	53	pom3c	5	9	3	60	20001
6	89	87	85	89	85	75	pom3d	5	9	3	60	501
-10	55	76	76	73	77	54	Wine	5	10	2	60	1600
-17	100	99	100	98	100	98	SS-L	5	11	2	60	1024
13	66	96	84	96	97	91	SS-O	6	11	2	60	973
-17	100	99	100	100	100	99	SS-P	7	11	2	60	1024
10	98	95	98	98	98	66	SS-X	4	11	2	60	86059
62	87	100	96	88	100	96	SS-T	6	12	2	60	5185
-30	63	98	82	97	98	98	SS-Q	6	13	3	60	2737
51	100	100	100	100	100	100	HSMGP	7	14	1	60	3457
-28	81	87	87	82	87	80	SS-R	7	14	2	60	3009
12	97	97	97	96	97	97	SS-V	6	16	2	60	6841
-5	54	70	66	54	99	53	SS-W	7	16	2	60	65537
15	100	100	100	100	100	100	X264-AM	7	16	1	60	1153
66	99	99	99	99	99	99	SS-M	7	17	3	60	865
9	-5	81	50	58	81	46	SS-N	6	17	2	60	53663

Table 4: EZR Performance comparison for medium datasets

To be more specific, Table 6 aggregates EZR’s performance across all 60 datasets used in this study. The first column shows the number of datasets and its corresponding proportion relative to the full datasets where EZR reaches a given performance threshold. The second column indicates the minimum percentage efficiency (relative to the best-performing answer) that EZR achieved in those cases. Notably, in 55 out of all 60 datasets (92%), EZR was able to find a configuration yielding at least 70% of the performance of the best-known method. Even under stricter criteria, EZR achieved at least 90% of the best-known performance in 46 datasets (77%), demonstrating strong label efficiency and practical competitiveness.

To provide statistical rigor, we incorporated the Scott-Knott (SK) test, a hierarchical clustering procedure designed for multiple treatment comparisons in software engineering research[57]. SK recursively partitions treatments into statistically distinct ranks by maximizing between-group differences while minimizing within-group variance. Unlike pairwise tests that suffer from multiple comparison problems, SK produces non-overlapping groups where treatments within the same rank are statistically indistinguishable. We conducted two SK analyses with different confidence thresholds. As shown in Table 7, using a stringent threshold(epsilon), EZR does not always achieve the top statistical rank occupied by fully supervised methods, as expected given EZR’s severe label constraints. However, with a relaxed

AsIs	LR	RF	SVR	ANN	LGBM	EZR	data	x*	x	y	budget	rows
-20	75	62	57	63	56	60	coc1000	5	20	5	60	1001
60	97	100	97	97	100	99	SS-U	8	21	2	378	4609
20	94	96	87	91	96	93	xomo-flight	7	27	4	810	10001
13	96	97	86	86	97	90	xomo-ground	7	27	4	810	10001
-6	92	95	93	93	95	93	xomo-osp	7	27	4	810	10001
10	92	90	82	85	91	83	xomo-osp2	8	27	4	810	10001
-1	69	74	75	71	85	66	SQL-AM	11	39	1	382	4654
-5	93	98	98	84	98	98	billing10k	12	88	3	810	10001
-4	97	97	97	97	97	97	Scrum100k	15	124	3	8010	100001
-19	95	95	95	95	95	95	Scrum10k	12	124	3	810	10001
10	95	95	95	95	95	86	Scrum1k	8	124	3	90	1001
9	99	99	99	99	99	99	FFM-125	8	128	3	810	10001
24	100	100	100	93	100	97	FFM-250	12	256	3	810	10001
11	95	95	95	93	95	95	FFM-500	13	510	3	810	10001
11	93	93	93	93	93	93	FM-500-1	11	511	3	810	10001
22	97	97	97	97	97	97	FM-500-2	11	511	3	810	10001
9	95	95	95	95	95	95	FM-500-3	9	513	3	810	10001
0	97	97	97	96	97	96	FM-500-4	14	517	3	810	10001
17	86	94	94	86	94	94	FFM-1000	11	1044	3	810	10001

Table 5: EZR Performance comparison for heavy datasets

# Datasets	Relative Efficiency
55 / 60 = 92%	$\geq 70\%$
52 / 60 = 87%	$\geq 80\%$
46 / 60 = 77%	$\geq 90\%$

Table 6: EZR Performance across all 60 datasets.

threshold(10% of the best achieved performance), EZR consistently clustered within the same statistical rank as state-of-the-art methods in the **majority** of datasets. This demonstrates that while EZR may not always match the absolute best performer under strictest criteria, the performance gap is statistically negligible under wider thresholds, substantiating our claim that EZR "consistently achieves near-optimal performance."

	Light(22)	Medium(19)	Heavy(19)
Tight Threshold	11/22	6/19	10/19
Wider Threshold	19/22	12/19	17/19

Table 7: On how many datasets in each group does EZR perform statistically indifferent compared to the best achieved performance?

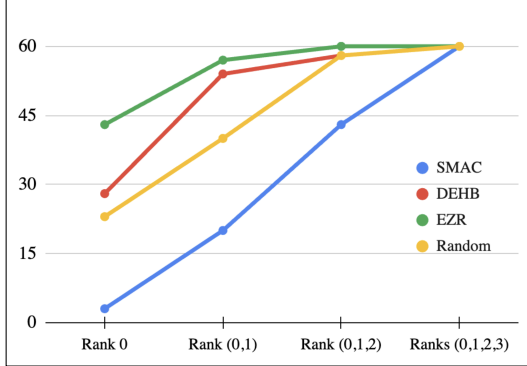


Figure 3: Among 60 datasets, how many times each method has statistically better performance, using 50 labels?

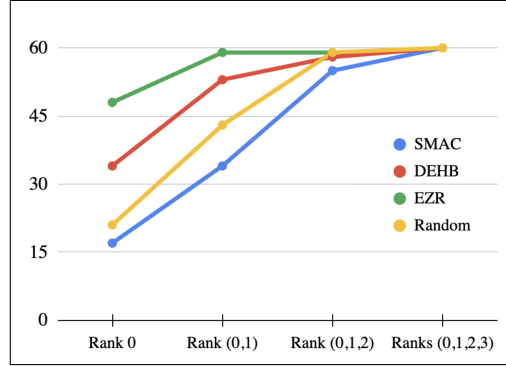


Figure 4: Among 60 datasets, how many times each method has statistically better performance, using 200 labels?

5.1.2. Optimization Baselines

To directly evaluate EZR against established optimization algorithms, we compared EZR against two state-of-the-art optimizers, SMAC and DEHB, under two fixed evaluation budgets:

- Labeling Budget = 50
- Labeling Budget = 200

All methods (EZR, SMAC, DEHB, and Random) were restricted to the same number of objective evaluations to ensure fairness.

Figures X and Y summarize the statistical rankings across the 60 datasets for budgets of 50 and 200 evaluations, respectively. The y-axis reports how many datasets each method was statistically indistinguishable from the top rank. Under the stricter budget of 50 evaluations, EZR achieves the top statistical rank in substantially more datasets than SMAC and DEHB. EZR is ranked in the best statistical group for nearly all datasets, while SMAC lags significantly under this tight budget. DEHB performs better than SMAC but remains below EZR in the number of top-ranked outcomes. With a larger budget of 200 evaluations, performance gaps narrow, as expected. All methods improve with additional evaluations. However, EZR continues to match or exceed the state-of-the-art methods across most datasets, remaining statistically competitive in nearly all cases.

These results show that EZR performs strongly under strict labeling budgets, consistently matching or outperforming SMAC and DEHB across the

majority of datasets. Even with only 50 evaluations, EZR frequently achieves statistically superior or indistinguishable performance compared to state-of-the-art optimizers. Increasing the budget to 200 evaluations reduces performance differences across methods, but EZR remains competitive.

Looking back to RQ1, we say:

Answering RQ1: Effectiveness on optimization. *The results demonstrate that EZR consistently achieves near-optimal performance across a wide range of datasets while using only a fraction of the labeling effort required by fully supervised models. This is a clear answer to RQ1; **EZR is an effective optimizer under label-scarce conditions, rivaling or exceeding state-of-the-art regression and optimization methods despite its minimal data requirements.** These findings reinforce the **Maximum Clarity Heuristic** introduced earlier—showing that, in complex software engineering optimization tasks, strong results can be achieved with less data when guided by clear and efficient heuristics.*

5.2. To what extent are EZR’s results explainable?

Understanding the outcomes of optimization in software engineering requires not only identifying optimal solutions but also interpreting *why* those solutions are preferred. This calls for XAI techniques that can provide insight into both the global behavior of the model and its local decision processes. RQ2 and RQ3 investigate the extent to which EZR provides explainable outputs compared to established XAI methods. Explainability is inherently multi-faceted: an explanation should be understandable to humans while also faithfully reflecting the underlying predictive signals. To capture both dimensions, we evaluate EZR using two complementary approaches. First, we present a subjective comparison of EZR’s local and global explanations against widely used methods. Second, we conduct an objective evaluation, where we test whether the feature-importance rankings produced by each explanation method can guide effective feature selection and improve downstream optimization. Together, these analyses provide a holistic assessment of EZR’s explainability.

5.2.1. Explanations in Practice: What Practitioners See

Explainability is particularly critical in software engineering, where developers, project managers, and stakeholders must understand model behavior

to justify decisions and ensure accountability. Without clear explanations, even high-performing models may face resistance in practice [63]. Explanations can be understood at two levels, *global explanations*, which reveal overall feature importance across the dataset, and *local explanations*, which clarify why the model produced a specific outcome for an individual instance. In the following, we present both global and local outputs of EZR and compare them against established methods.

Global Explanation: For global explanations, we adopt the permutation importance framework, a widely used model-agnostic technique for quantifying feature contributions. Following Alcaide et al. [64], who demonstrated its utility for configuration-error detection, we use this approach to benchmark EZR. Figure 5 presents the resulting permutation importance values (y-axis). A large degradation in performance indicates that a feature is highly informative, while negative importance values suggest that a feature’s original distribution may add noise or reduce predictive accuracy.

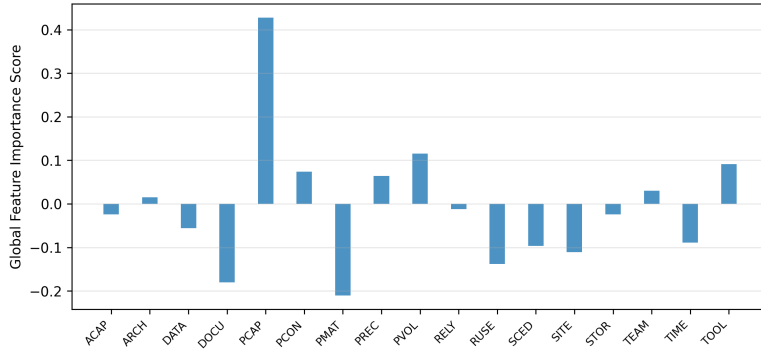


Figure 5: COC1000 Feature importance via permutation

As discussed earlier, to solve optimization problems, our method **EZR** constructs an interpretable decision model using only a small number of labeled examples. Figure 6 presents a Parzen-style decision tree constructed by EZR⁵, where each row corresponds to a node that recursively splits the configuration space. Internal nodes divide the data based on feature thresholds (e.g., $TEAM \leq 5$), and leaf nodes are indicated by a semicolon (;).

⁵Extra details are omitted for brevity. For the whole tree, refer to https://github.com/amiiralii/Minimal-Data-Maximum-Clarity/tree/main/paper_materials

win	n	
6	32	
12	30	STOR <= 5
13	28	TEAM <= 5
20	25	TEAM <= 4
22	23	PREC <= 5
34	2	TEAM <= 1 ;
21	21	TEAM > 1
41	2	PVOL <= 2 ;
18	19	PVOL > 2
65	4	PCON <= 1
76	2	ACAP > 4 ;
55	2	ACAP <= 4 ;
6	15	PCON > 1
48	2	PREC <= 2 ;
-1	13	PREC > 2
8	2	RUSE <= 2 ;
-2	11	RUSE > 2
0	9	PVOL <= 4
16	2	PCON > 4 ;
-5	7	PCON <= 4
17	2	RUSE > 5 ;
-14	5	RUSE <= 5
-9	2	DOCU <= 2;
-17	3	DOCU > 2;
-11	2	PVOL > 4 ;
-3	2	PREC > 5 ;
-45	3	TEAM > 4 ;
-1	2	TEAM > 5 ;
-79	2	STOR > 5 ;

Figure 6: EZR tree output showing impurity-based splits and performance metrics at each node.

Each node includes the number of configurations it contains (n) and the *win* score of the median performance in terms of $d2h$. As an example, the most optimal leaf in this tree corresponds to a branch with only 2 configurations (out of all those intelligently chosen ones) and a win score of 76%. To identify high-performing configurations in practice, we would pass any test instances through the tree and observe which branch they fall into. We then inspect the configurations in that branch and label the top-ranked ones to determine which yields the best actual $d2h$. Also, based on the EZR tree and by using MDI (Equation 2), we can extract feature importance scores. This use case will be discussed in section 4.2.2.

Local Explanation To illustrate local explanations of common baselines, we use a sample test instance from the COC1000 dataset (Table8) and predict its $d2h$ value with a LightGBM model. For this instance, we generate explanations using three widely adopted techniques: **LIME**, **SHAP**, and **BreakDown**. In contrast, EZR provides local explanations natively. Its decision tree structure allows tracing the decision path for a given configuration, thereby revealing why the model identifies it as promising or not.

ACAP 2	ARCH 5	DATA 3	DOCU 1	PCAP 3	PCON 1	PMAT 3	PREC 5	PVOL 3
RELY 1	RUSE 2	SCED 2	SITE 4	STOR 3	TEAM 3	TIME 5	TOOL 3	

Table 8: Feature values for the selected test instance

Starting with LIME’s results available in Figure 7, the surrogate model attributes the prediction (0.50 vs. true 0.45) mainly to $SCED \leq 2$ and $ACAP \leq 2$, both of which push the value upward. In contrast, $2 < PMAT \leq 4$ exerts the strongest downward effect, indicating that moderate process maturity improves the configuration. Considering smaller contributions from other features, together these attributions explain the adjustment from the local baseline to the final prediction.

Moving on to SHAP, Figure 8 shows the output of SHAP’s waterfall results. It begins at the baseline expectation $E[f(X)] = 0.454$ and then sees how each feature nudges the prediction down to $f(x) = 0.443$. The largest driver is TIME, which strongly reduces the $d2h$ value, while DOCU raises it slightly. Other factors such as PMAT, ARCH, and ACAP each provide small downward contributions, whereas TEAM nudges the value upward. Overall, SHAP highlights TIME and DOCU as the dominant influences for

True target = 0.45, Model prediction = 0.50

LIME Explain:

SCED ≤ 2.00 : +0.017 ($\uparrow$)
 ACAP ≤ 2.00 : +0.010 ($\uparrow$)
 $2.00 < \text{PMAT} \leq 4.00$: -0.006 ($\downarrow$)
 $4.00 < \text{TIME} \leq 5.00$: +0.006 ($\uparrow$)
 $4.00 < \text{PREC} \leq 5.00$: +0.003 ($\uparrow$)
 DOCU ≤ 2.00 : +0.003 ($\uparrow$)

Figure 7: LIME explanation output

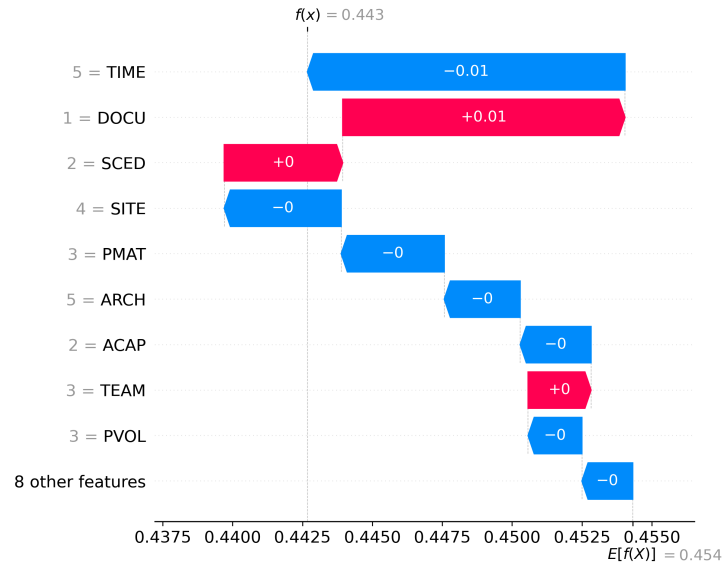


Figure 8: SHAP explanation output

this instance.

For BreakDown, Figure 9, the prediction builds sequentially from the baseline 0.454 to 0.500. The largest contributors are DOCU, SCED, and TIME, with smaller additions from ACAP, SITE, and others. This stepwise view highlights both the dominant drivers and how their cumulative effects produce the final prediction.

For EZR's local explanation, we trace the decision path from the root to the final leaf of the tree for the given instance (Figure 10). The objective is not to monitor incremental prediction changes but to identify the leaf that groups together configurations sharing similar characteristics with the

instance. In this case, the path terminates in a leaf containing only two configurations, providing a cohort-based rationale: *this configuration behaves like those cases*. Our implementation also exposes the instances within each leaf along with summary statistics (e.g., mean $d2h$, variance, win percentage, Impurity), enabling direct inspection of how comparable cases performed in practice. It is important to note that this path involves only six features, while the full EZR tree uses nine drawn from seventeen. This reduction underscores EZR’s ability to highlight only the most informative attributes, which aligns with our **Maximum Clarity Heuristic** that complex tasks can often be explained with less data.

Having presented the global and local outputs of EZR alongside established methods such as LIME, SHAP, and BreakDown, we now turn to a subjective comparison of their explanatory value. To structure this assessment, we draw on the explanation triggers and user-goal framework (Table 9), which captures the types of questions practitioners typically pose when interacting with AI systems. By examining how well each method addresses these triggers, we can evaluate their relative clarity, actionability, and alignment with stakeholder needs.

Building on prior work advocating the integration of causal inference into software engineering [66], we also evaluate **EZR** through Pearl’s Ladder of Causation [47], which distinguishes three levels of reasoning; **seeing** (associations), **doing** (interventions), and **imagining** (counterfactuals). Attribution methods largely remain on the first rung, providing associational insight by decomposing a prediction into additive contributions ($P(Y|T)$) but without modeling interventions or alternative realities. EZR, by contrast, spans all three rungs. At the associational level, its global tree structure and permutation importance summarize which features matter overall. At the inter-

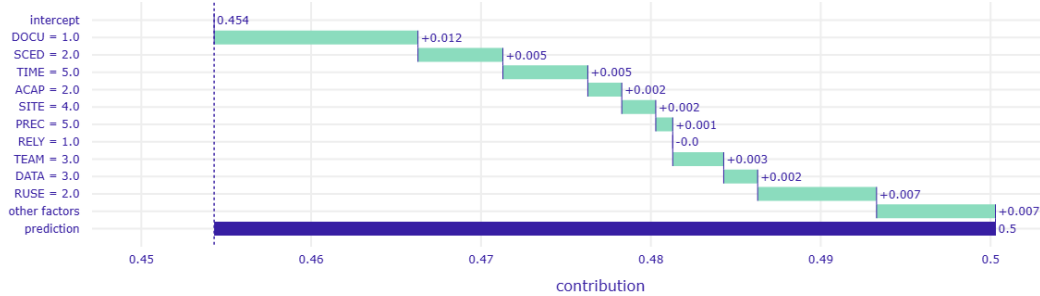


Figure 9: BreakDown explanation output

win	n	
6	32	
12	30	STOR <= 5
13	28	TEAM <= 5
20	25	TEAM <= 4
22	23	PREC <= 5
21	21	TEAM > 1
18	19	PVOL > 2
65	4	PCON <= 1
55	2	ACAP <= 4 ;

Figure 10: Highlighted decision path in the EZR tree

ventional level, EZR natively supports “what-if” reasoning. Each split is a concrete threshold, so modifying a feature and retracing the path yields a new leaf (cohort) and updated outcome summaries, directly operationalizing the do-operator ($P(Y|do(T))$). Finally, at the counterfactual level, minimal edits to feature values identify the closest alternative leaf where the decision would differ, enabling users to ask not only “what if x were different?” but also “why didn’t it do z ?”. Viewed through the practitioner triggers in Table 9, this causal mapping strengthens our subjective evaluation: EZR addresses global needs (“How does it work?”), local reasoning (“What did it just do?”), and counterfactual queries (“what-if/why”) in a unified and deterministic framework. In doing so, it delivers explanations that are interpretable, consistent, and operational.

To further illustrate EZR’s explanatory capability, we perform a walk-through using the sample configuration from Table 7. We evaluate how well the cohort-based explanation addresses each practitioner question outlined in Table 8.

1. **How do I use it?** Feed the instance into the EZR tree (Figure 3) to trace its decision path. Looking at the win rate of the leaf the configuration ended up in, you can decide how good the instance is.
2. **How does it work?** EZR splits the configuration space using the most influential features first, those appearing near the root or repeatedly across branches. The MDI score quantifies this influence, offering a global view of which features consistently shape decisions. In practice, this provides a quick, interpretable summary of what matters most in the system.

Trigger	User/Learner’s Goal	Rung in ladder of causation
How do I use it?	Achieve the primary task goals	—
How does it work?	Satisfaction from achieving global understanding	Association
What did it just do?	Satisfaction from understanding local decision	Intervention
What does it achieve?	Understand the system’s functions and use	Intervention
What will it do next?	Trust based on observability and predictability	Intervention
How much effort will this take?	Effectiveness in achieving primary goals	Intervention
What do I do if it gets it wrong?	Avoid mistakes	Counterfactual
How do I avoid the failure modes?	Mitigate errors	Counterfactual
What would it have done if x were different?	Resolve curiosity through counterfactual understanding	Counterfactual
Why didn’t it do z ?	Understand the rationale behind a local decision	Counterfactual

Table 9: Common explanation triggers and associated user goals in XAI systems [65] with corresponding rung in the ladder of causation [47]

3. **What did it just do?, What does it achieve?** EZR evaluated the instance by following a path through key splits, for example, $TEAM \leq 4$ suggests a smaller, likely more cohesive team, or $PREC \leq 5$ implies moderate process flexibility. Based on these and other decisions(Figure 10), the configuration was assigned to a cohort with a win score of 55, meaning it performs reasonably well but not among the top-tier options. In practice, this points to a viable but improvable setup for project planning.
4. **What will it do next?** Based on the current cohort’s win score of 55 (vs. a peak of 76 in the tree), EZR infers that this configuration is unlikely to be optimal. Rather than labeling or deploying it, the system recommends exploring other configurations with higher potential. Also, in order to end up in the best known leaf, EZR suggests increasing ACAP, corresponding to having more skilled analysts for the project.
5. **How much effort will this take?** EZR requires minimal effort. It simply routes the new configuration through the tree in milliseconds. If the outcome is suboptimal, users can review the decision path, identify the feature that steered the configuration into a low-performing branch (e.g., $ACAP < 4$), and tweak that setting to redirect it toward a better-performing cohort. Thus, no retraining or full model inspection needed.
6. **What do i do if it gets wrong?, How do i avoid the failure**

modes? Practitioners can mitigate risks in three ways. First, remain cautious of dataset biases. If most high-performing samples share specific traits (for instance, favoring small teams if most high-performing examples come from solo projects), EZR’s tree may overemphasize them; inspecting second-best cohorts can reveal more balanced alternatives. Second, revisit the labeling budget. Although we adopted values recommended by prior work, some problems may require more labels for stability, while in other cases fewer may suffice. So adaptive adjustments are encouraged. Finally, when a configuration is assigned to a leaf but lies far from that leaf’s centroid (compared to its peers), the cohort rationale may be unreliable. In such cases, users should treat EZR’s recommendation as tentative and seek validation from nearby leaves or additional labeling.

7. **What would it have done if x were different?** EZR enables easy what-if reasoning. For our sample configuration, increasing ACAP (Analyst Capability) would redirect it from a lower performing branch (win = 55) to a higher one (win = 76). This suggests that boosting analyst expertise, perhaps through targeted hiring or internal training programs, could significantly improve project outcomes. Such actionable insight helps decision-makers connect model recommendations to real-world interventions.
8. **Why didn’t it do z ?** Answering counterfactual questions is straightforward with EZR, thanks to its transparent decision tree. For example, “Why isn’t this configuration among the best?” Because its ACAP (Analyst Capability) is too low, pushing it into a lower-performing branch. On the other hand, a higher-scoring configuration satisfies key thresholds such as $STOR \leq 5$ indicating sufficient storage resources, $TEAM \leq 4$ reflecting a cohesive small team structure, $PREC \leq 5$ suggesting moderate flexibility in development processes (Precedentedness), $PCON \leq 1$, implying low risk of personnel change (Personnel Continuity), and $ACAP > 4$, showing strong analytical expertise on the team. These thresholds reveal not only how the model makes decisions, but also why certain configurations succeed, which allows stakeholders to reason through and improve suboptimal cases with concrete, domain-aligned actions.

In summary:

Answering RQ2: Comparison to standard XAI. *EZR pro-*

*vides explanations that go beyond attribution-based methods like LIME, SHAP, or BreakDown. By mapping instances to concrete cohorts with summary statistics, it delivers stable, rule-based, and actionable insights rather than abstract contributions. Within Pearl’s Ladder of Causation, EZR spans all three rungs (associations through global feature importance, interventions through threshold-based what-if analysis, and counterfactuals through alternative leaf assignments) whereas attribution methods remain confined to associations. Consistent with our **Maximum Clarity Heuristic**, EZR achieves this explanatory power while relying on fewer, more informative features. In doing so, EZR aligns closely with practitioner triggers (Table 9), offering explanations that are interpretable, consistent, and directly useful in real-world SE tasks.*

5.2.2. Explanations in Action: What the Features Reveal

In order to test whether explanations aid optimization in practice, we treat each method’s feature importance scores as a feature selector and measure downstream utility. Inspired by XAI’s actionability principle, we convert scores into a compact subset, train an independent learner on those features, and compare against all-features and common SE feature ranking tools (SHAP, ReliefF, ANOVA). The idea is simple. If an explanation is faithful, its subset should help learners achieve lower the $d2h$ or higher win rates than baselines. This setup makes explanation quality a falsifiable claim about utility beyond the method that produced it.

Table 10 evaluates whether explanation-driven feature subsets improve downstream optimization. We compare three commonly used methods in the SE domain, ReliefF, SHAP, ANOVA, alongside using all features, and EZR. Each of the methods is paired with the LightGBM regressor across all datasets⁶. Cells report the median $win(d2h)$ for the feature selector and dataset. The X column gives the total number of features in the dataset; X^* is the number of features selected (for treatment "all" $X = X^*$). “Budget” records how many labels EZR used to compute feature importances. (It is either 150 or 40% of the training set, whichever is smaller) We have also used color coding to aid comparison. White marks the per-dataset best (there is

⁶We have also executed this experiment using other regressors. Refer to this link: https://github.com/amiiralii/Minimal-Data-Maximum-Clarity/tree/main/paper_materials

at least 1 white cell in each row); Blue reaches $\geq 90\%$ of the referenced optimal; Yellow and red are $\geq 75\%$ and $\leq 75\%$ of the referenced optimal, respectively. Within a given regressor block, read *across a row* to compare feature selectors for the same dataset. EZR should be interpreted jointly with the Budget. Cases where small subsets trained from limited labels still land in the white/blue region indicate explanations that capture optimization-relevant signal.

Figures 11 and 12 present an aggregated view of Table 10, summarizing outcomes across all treatments. To evaluate feature-reduction efficiency, datasets were grouped into five bins based on feature count (≤ 6 , <20 , <100 , <500 , and $500+$). With 60 datasets and 5 regressors, this results in 300 treatments distributed across these bins. Each point in the figures shows the proportion of treatments per each method in a given bin that achieved at least 75% (Figure 11) or 90% (Figure 12) of the referenced optimal performance.

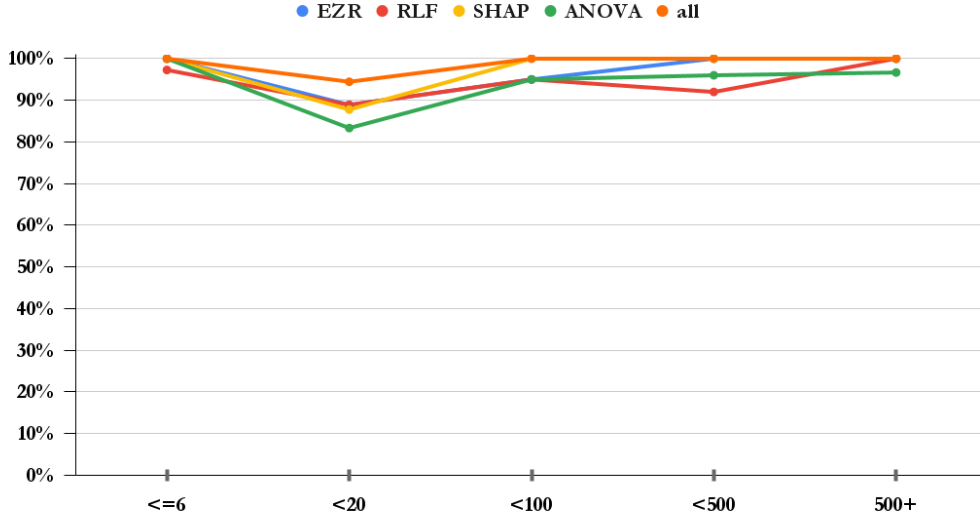


Figure 11: In what portion of each dataset group does each method achieve 75% of the referenced optimal or closer?

Overall, the aggregated results reveal two key trends. In the 75% threshold analysis (Figure 11), the performance of all methods largely converges, with little separation across buckets. This “blurring” effect indicates that EZR is just as effective as other approaches while relying on far fewer labels. In contrast, the 90% threshold analysis (Figure 12) shows clearer differences,

RLF	SHAP	EZR	anova	all	data	x*	x	y	budget	rows
93	93	93	93	93	SS-A	3	3	2	150	1344
84	84	84	84	84	SS-B	2	3	2	80	207
84	84	84	84	84	SS-C	3	3	2	150	1513
77	77	77	77	77	SS-D	3	3	2	75	197
97	97	97	97	97	SS-E	3	3	2	150	757
98	98	98	97	98	SS-F	2	3	2	75	197
93	91	91	91	93	SS-G	2	3	2	75	197
80	99	99	99	99	wc-1	2	3	1	75	197
94	99	99	99	99	wc-2	2	3	1	75	197
94	99	99	99	99	wc-3	2	3	1	75	197
100	100	100	100	100	SS-H	2	4	2	100	260
99	99	99	99	99	SS-I	5	5	2	150	1081
80	80	80	79	79	auto93	3	5	3	150	399
66	66	66	66	66	HCI-hard	5	5	3	150	10001
100	100	100	100	100	HCI-easy	4	5	3	150	10001
96	95	96	95	96	SS-J	5	6	2	150	3841
96	96	96	96	96	SS-K	5	6	2	150	2881
100	100	100	100	100	SS-S	5	6	2	150	3841
100	100	100	100	100	rs-1	5	6	1	150	3841
100	100	100	100	100	rs-2	5	6	1	150	3841
94	94	94	94	94	sol-1	5	6	1	150	2867
100	100	100	100	100	wc-4	4	6	1	150	2881
95	95	95	95	95	Apache-AM	4	9	1	75	193
87	90	88	90	88	pom3a	7	9	3	150	20001
85	87	87	85	87	pom3b	6	9	3	150	20001
73	68	71	73	73	pom3c	7	9	3	150	20001
88	88	76	85	82	pom3d	7	9	3	150	501
77	77	80	77	77	Wine	6	10	2	150	1600
98	99	99	99	99	SS-L	6	11	2	150	1024
91	99	96	99	96	SS-O	6	11	2	150	973
98	99	98	99	99	SS-P	6	11	2	150	1024
60	59	41	59	97	SS-X	1	11	2	150	86059
86	99	100	99	100	SS-T	5	12	2	150	5185
93	97	97	97	98	SS-Q	7	13	3	150	2737
95	100	100	100	100	HSMGP	6	14	1	150	3457
87	81	82	82	85	SS-R	7	14	2	150	3009
94	95	63	57	96	SS-V	4	16	2	150	6841
74	74	59	74	99	SS-W	7	16	2	150	65537
100	100	100	100	100	X264-AM	6	16	1	150	1153
100	100	100	99	100	SS-M	7	17	3	150	865
81	81	81	81	81	SS-N	8	17	2	150	53663
59	62	53	52	58	coc1000	9	20	5	150	1001
95	96	96	93	100	SS-U	7	21	2	150	4609
98	98	93	98	98	xomo_flight	7	27	4	150	10001
92	92	90	90	92	xomo_grnd	7	27	4	150	10001
94	95	95	95	95	xomo_osp	7	27	4	150	10001
87	88	82	82	88	xomo_osp2	8	27	4	150	10001
77	75	71	64	85	SQL-AM	10	39	1	150	4654
87	91	92	74	94	billing10k	7	88	3	150	10001
75	75	81	75	97	Scrum100k	7	124	3	150	100001
82	88	92	88	97	Scrum10k	8	124	3	150	10001
83	89	86	83	89	Scrum1k	7	124	3	150	1001
81	99	93	81	99	FFM-125	9	128	3	150	10001
81	97	89	97	97	FFM-250	8	256	3	150	10001
95	93	93	95	95	FFM-500	8	510	3	150	10001
81	96	87	96	96	FM-500-1	8	511	3	150	10001
86	91	93	91	97	FM-500-2	7	511	3	150	10001
95	95	95	87	95	FM-500-3	9	513	3	150	10001
97	97	93	96	97	FM-500-4	9	517	3	150	10001
82	94	94	71	100	FFM-1000	7	1044	3	150	10001

Table 10: EZR Performance as a feature selection tool. White cells show the best performance; blue and yellow cells mark EZR answers that achieve at least 90% and 75% of the best;

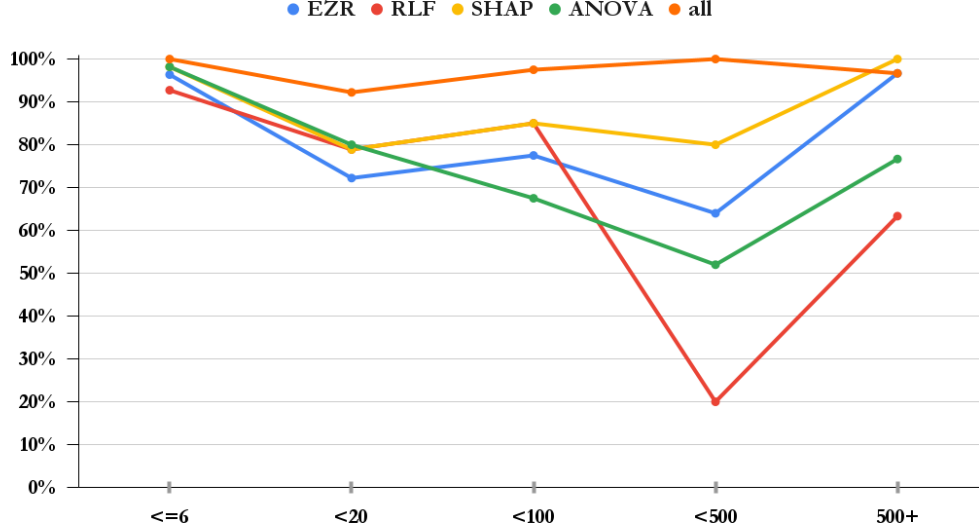


Figure 12: In what portion of each dataset group does each method achieve 90% of the referenced optimal or closer?

particularly in the larger-feature buckets, where all methods fall short. The absence of such gaps at the 75% level suggests that a substantial portion of results lies between 75–90% of optimal performance. Importantly, only SHAP consistently outperforms EZR in these higher thresholds, yet it does so with full supervision, whereas EZR achieves comparable performance using only 150 labels.

In summary:

Answering RQ3: Practical utility of explanations. *EZR’s explanation-driven feature rankings substantially improve downstream optimization, performing on par with or better than established feature selection methods under most conditions. Although SHAP occasionally yields stronger results in high-dimensional settings, EZR achieves comparable performance while relying on far fewer labels. This outcome reinforces the Maximum Clarity Heuristic: in complex software engineering tasks, effective explanations and strong optimization performance can be achieved with less data, not more.*

6. Discussion

Our results suggest that EZR’s remarkable label efficiency and robust performance stem from two core mechanisms: selective sampling that skips noisy or irrelevant configurations, and the use of a compact decision tree to guide further exploration. By focusing only on those samples most likely to yield improvements, EZR avoids the “needle-in-a-haystack” problem that plagues exhaustive search, and then leverages a small decision tree to rapidly identify promising directions.

6.1. Why EZR Works

1. **Noise avoidance and relevance filtering.** Rather than uniformly sampling the entire configuration space, EZR’s warm-start and active learning steps label the “corners” of the configuration space, the extreme parameter combinations that are most likely to lie on the Pareto front. By doing so, it avoids wasting budget on noisy or redundant regions and concentrates sampling where performance variation is highest. This strategy is well-suited to software engineering data, which often resides on low-dimensional manifolds. Lustosa and Menzies’s Dimensionality Reduction Ratio (DRR) study shows that a small fraction of intrinsic dimensions captures nearly all the variability in software engineering performance metrics [25].
2. **Lightweight surrogate modeling.** The decision tree induced from these select labels is necessarily small. Its depth and branching factor are both bounded by the number of samples. Such a tree can be traversed in $O(\log n)$ time to suggest tweaks that push toward better performance, and its human-readable structure offers immediate interpretability.
3. **Sensitivity to labeling budget.** To assess how sensitive EZR is to the chosen labeling budget, we repeated the same optimization protocol under alternative budgets and compared the resulting performance distributions to our default setting. For *light* and *medium* datasets, we varied the absolute labeling budget $\{10, 25, 50, 100, 200\}$ and for *heavy* datasets, we varied the budget as a fraction of the training set $\{1\%, 5\%, 10\%, 20\%, 40\%\}$. For each dataset and each budget treatment, we repeated optimization 20 times using different random seeds, producing a distribution of *win* scores per treatment. We then compared each treatment’s *win* distribution against the distribution obtained with

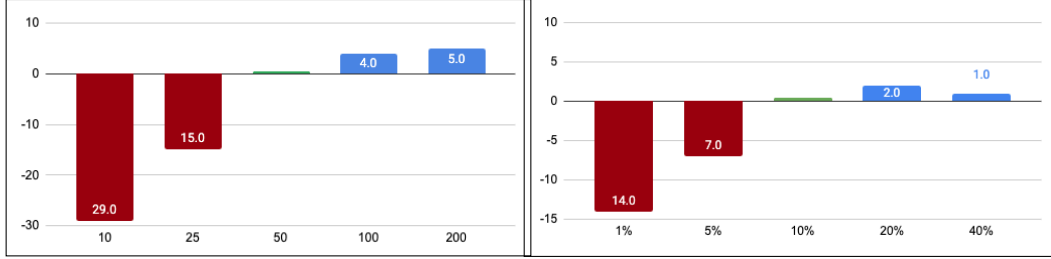


Figure 13: Sensitivity on light and medium datasets (41 in total). Red indicates the number of datasets performing significantly worse than our chosen budget, if we use too few labels (fewer than 50). The blue columns show the number of datasets performing significantly better than our chosen budget (using higher budgets), offering very little additional benefit.

Figure 14: Sensitivity on heavy datasets (19 in total). Red indicates the number of datasets performing significantly worse than our chosen budget, if we use too few labels (fewer than 50). The blue columns show the number of datasets performing significantly better than our chosen budget (using higher budgets), offering very little additional benefit.

our selected budget (50 labels for light/medium and 10% for heavy), and counted how many datasets (out of 60) were *significantly different* from the selected budget. The resulting counts are summarized in Figure 13 (light/medium) and Figure 14 (heavy). Overall, the sensitivity analysis shows a clear pattern. Budgets below the selected defaults are frequently and significantly worse, whereas increasing the budget beyond the selected defaults rarely produces statistically distinguishable improvements.

6.2. Limitations and Caveats

- **Dataset Characteristics:** EZR’s efficiency hinges on the idea that high-quality configurations can be (at least partially) ordered by their performance. In problems where the objective landscape is highly irregular or multimodal with many local optima that offer conflicting trade-offs, the sampling strategy may fail to isolate the true Pareto front, leading to suboptimal recommendations. Moreover, because EZR focuses on the most promising areas, it may systematically overlook rare but critical configurations that lie outside the main performance clusters. In safety-critical contexts, explicit mechanisms to ensure coverage of edge-case scenarios may be necessary.
- **Problem setting:** As the number of available parameter values grows, the size and complexity of the induced decision tree can rise, demand-

ing more labels to maintain predictive accuracy. More complex feature configuration spaces may require additional strategies to keep EZR efficient. Also, EZR assumes a fixed configuration space and a single-shot labeling process. In dynamic or streaming environments, where system behavior drifts over time or new options appear, EZR would need extensions for incremental learning and adaptive re-budgeting to remain effective.

By highlighting both the theoretical rationale and practical boundaries of EZR, we provide practitioners with a clear understanding of when and how this lightweight, interpretable heuristic can be most effectively applied and when more exhaustive or specialized methods remain necessary.

6.3. Threats to validity

To ensure rigor and transparency, we’ve examined key factors that might affect our results and rerun experiments under varied settings to confirm their stability. Any conclusions should be interpreted with the following potential limitations in mind:

1. **Conclusion validity:** Conclusion validity threats arise when the observed effects may be due to random variation or the choice of specific analysis procedures rather than reflecting true performance differences. EZR introduces two sources of randomness. The initial warm-start selection of rows and the sampling performed within the active learning loop. To reduce the influence of any single random sequence, we repeated every experiment 20 times, each with a unique random seed, and applied a fresh 80/20 train/test split for each run. Reported results are the mean over these 20 trials, ensuring that our findings are robust to stochastic variation and not artifacts of chance.
2. **Internal Validity:** Internal validity threats concern whether the performance differences observed are attributable to our method rather than to uncontrolled factors. We minimized this risk by employing consistent experimental protocols across all treatments. Identical dataset splits, identical evaluation metrics, and fixed labeling budgets for different treatments. All baseline models were implemented using well-established libraries with default or widely accepted hyperparameters, ensuring fair and reproducible comparison.

A deliberate difference between EZR and the baselines is label availability, as label efficiency is central to our research questions. In the RQ1 experiment, for light and medium datasets, EZR’s labeling budget

was fixed at 60 examples, derived from performing 50 active learning iterations to expand the decision tree, followed by selecting the top 10 candidates from the unlabeled pool for final evaluation. This choice of 50 iterations follows the finding by Ganguly and Menzies [58] that most datasets reach state-of-the-art performance at this point. For heavy datasets, which embed more complex patterns, we evaluated budgets of 1%, 5%, 10%, 15%, and 20% of the training data. Performance gains beyond 10% were negligible. Accordingly, we fixed the budget for heavy datasets at 10% of the training set.

Selecting representative baselines is also critical to avoid biased comparisons. We chose one strong performer from each major regression family. Linear regression, decision trees, bagging (Random Forests), boosting ensembles (LightGBM), neural networks, and support vector machines, ensuring that our predictive model comparisons span both simple and highly optimized learners. For feature ranking, we included SHAPley values (an explanation-driven approach), ReliefF (a high-performing filter method), ANOVA (a classic statistical test), and an all-features baseline to capture exhaustive selection. While no single baseline consistently outperforms all others across every dataset, this diversity provides a broad and rigorous context against which to evaluate EZR.

3. **Construct validity:** Construct validity threats arise when the chosen evaluation measures do not accurately capture the intended study objectives. We assess optimization performance using the $d2h$ score, from which we derive a “win rate” across methods. To verify that our conclusions are not dependent on this specific metric, we repeated all experiments using the Chebyshev distance [21, 67]. While absolute scores differed, the relative rankings and overall conclusions remained stable.

In addition, because statistical rank-based tests such as Scott Knott do not convey how close methods are in absolute terms, we introduced a relative performance metric. This metric quantifies how near EZR comes to the best-known solution achieved by a fully supervised state-of-the-art method, thereby directly reflecting the trade-off between accuracy and labeling cost.

For feature selection, we evaluated predictive fidelity within the optimization task, acknowledging that these results may not transfer directly to other tasks such as classification or regression.

4. **External validity:** External validity threats concern the extent to which our findings generalize beyond the studied datasets. Real-world software engineering optimization problems vary widely in feature distributions, noise characteristics, and cost structures, which small benchmark collections may fail to capture. To address this, we evaluated EZR on 60 diverse datasets (far exceeding the dataset counts used in prior work) from the MOOT repository, which covers configuration, hyperparameter tuning, and process optimization scenarios. The MOOT repository was specifically developed to enhance the generality of search-based software engineering conclusions. It aggregates datasets from numerous recent SE and AI publications into a unified testbed that reflects a broad spectrum of real-world optimization challenges. Leveraging MOOT, therefore, strengthens the external validity of our study and increases the likelihood that our conclusions apply to other problem domains.

6.4. Future Work

A promising direction is to move beyond fixed labeling budgets by developing adaptive schemes that monitor convergence or model uncertainty at each step and automatically allocate or halt sampling when marginal gains wane. Complementing this, we plan to validate EZR on a broader array of software engineering benchmarks drawn from different repositories and use cases (e.g., build-system tuning, static-analysis parameterization, continuous-integration workflows) to test its robustness across practical scenarios. We also see value in embedding EZR as the front-end of hybrid optimization pipelines, using its rapid “good-enough” search to identify a promising region before handing off to more exhaustive global optimizers for fine-tuning. Finally, integrating EZR’s active learning strategy into automated machine learning frameworks, where it could drive both configuration and feature engineering choices, offers a path to end-to-end systems that balance interpretability, efficiency, and accuracy.

7. Conclusion

We have presented EZR, a simple yet effective framework for multi-objective optimization and explanation in software engineering. Instead of relying on large labeled datasets and complex models, EZR uses a small,

actively selected subset of configurations to build concise decision-tree explanations and surrogate predictors. Evaluated on over 60 diverse MOOT datasets, spanning configuration, hyperparameter tuning, and process optimization, EZR delivers near-optimal settings while querying only a fraction of the labels required by state-of-the-art methods.

First, in optimization, based on tables 3, 4, and 5, EZR achieves within 90% of the best-known performance in 77% of datasets (and greater than 70% in 92% of cases) using only 60 labels (or 10% of the data for large problems), versus full supervision for baselines. Its simplicity and label efficiency make it an attractive choice for rapid tuning in domains that currently lack systematic optimization tools. In practice, EZR is ideal when a “good-enough” setting is needed quickly and with minimal effort. However, because EZR does not guarantee the absolute optimum, critical or high-stake applications (e.g., healthcare or finance) may still require more exhaustive or specialized optimizers. In addition, we further compared EZR against established optimizers such as SMAC and DEHB. Despite its simplicity and lower labeling cost, EZR achieves competitive performance to these methods across a large fraction of datasets, reinforcing its competitiveness within the broader optimization literature.

Second, as a feature ranking tool, according to table 10, EZR matches or closely approaches the performance of established methods (e.g., SHAP, ReliefF, ANOVA) while using significantly fewer labels. This efficiency makes it ideally suited to scenarios where labeling costs are the primary constraint. At the same time, when marginal gains in predictive accuracy outweigh labeling effort, more aggressive selection techniques may be preferred. Moreover, our results indicate that predictive accuracy hinges more on the choice of regression model and dataset characteristics than on the specific feature ranking method.

Overall, EZR embodies the “less but better” principle. By carefully choosing minimal data, we achieve maximum clarity and strong performance. We hope this work encourages further exploration of lightweight, interpretable heuristics in automated software engineering.

References

- [1] T. Xu, X. Jin, P. Huang, Y. Zhou, S. Lu, L. Jin, S. Pasupathy, Early detection of configuration errors to reduce failure damage, in: Proc. OSDI 2016, 2016.

- [2] T. Chen, M. Li, Adapting multi-objectivized software configuration tuning, *Proc. ACM Softw. Eng.* (2024).
- [3] M. Sayagh, N. Kerzazi, B. Adams, F. Petrillo, Software configuration engineering in practice interviews, survey, and systematic literature review, *IEEE Trans. Softw. Eng.* (2018).
- [4] T. Xu, L. Jin, X. Fan, Y. Zhou, S. Pasupathy, R. Talwadker, Hey, you have given me too many knobs!: understanding and dealing with over-designed configuration in system software, in: *Proc. FSE 2015*, 2015.
- [5] S. Mühlbauer, F. Sattler, C. Kaltenecker, J. Dorn, S. Apel, N. Siegmund, Analysing the impact of workloads on modeling the performance of configurable software systems, in: *Proc. ICSE 2023*, 2023.
- [6] S. Majumder, J. Chakraborty, T. Menzies, When less is more: on the value of “co-training” for semi-supervised software defect predictors, *Empir. Softw. Eng.* (2024).
- [7] J. A. Pereira, M. Acher, H. Martin, J.-M. Jézéquel, G. Botterweck, A. Ventresque, Learning software configuration spaces: A systematic literature review, *J. Syst. Softw.* (2021).
- [8] X. Wu, W. Zheng, X. Xia, D. Lo, Data quality matters: A case study on data label correctness for security bug report prediction, *IEEE Trans. Softw. Eng.* 48 (2021).
- [9] A. K. Shakya, G. Pillai, S. Chakrabarty, Reinforcement learning algorithms: A brief survey, *Expert Syst. with Appl.* 231 (2023).
- [10] J. Bergstra, Y. Bengio, Random search for hyper-parameter optimization, *The journal of machine learning research* (2012).
- [11] Z. Li, M. Harman, R. M. Hierons, Search algorithms for regression test case prioritization, *IEEE Trans. Softw. Eng.* (2007).
- [12] M. Khatibsyarbini, M. A. Isa, D. N. Jawawi, R. Tumeng, Test case prioritization approaches in regression testing: A systematic literature review, *Inf. Softw. Technol.* (2018).
- [13] F. Hujainah, R. B. A. Bakar, M. A. Abdulgabbler, K. Z. Zamli, Software requirements prioritisation: a systematic literature review on significance, stakeholders, techniques and challenges, *IEEE Access* (2018).
- [14] Y. Zhang, M. Harman, S. A. Mansouri, The multi-objective next release problem, in: *Proc. GECCO 2007*, 2007.
- [15] A. V. Rezende, L. Silva, A. Britto, R. Amaral, Software project scheduling problem in the context of search-based software engineering: A systematic review, *J. Syst. Softw.* (2019).
- [16] F. Sarro, Search-based software engineering in the era of modern software systems, in: *Proc. IEEE Int. Conf. on Requirements Engineering*, 2023.

- [17] W. Mkaouer, M. Kessentini, A. Shaout, P. Koligheue, S. Bechikh, K. Deb, A. Ouni, Many-objective software remodularization using nsga-iii, ACM TOSEM (2015).
- [18] V. Nair, Z. Yu, T. Menzies, N. Siegmund, S. Apel, Finding faster configurations using flash, IEEE Trans. Softw. Eng. (2018).
- [19] M. Li, T. Chen, X. Yao, How to evaluate solutions in pareto-based search-based software engineering: A critical review and methodological guidance, IEEE Trans. Softw. Eng. (2020).
- [20] P. Chen, T. Chen, Promisetune: Unveiling causally promising and explainable configuration tuning, arXiv preprint arXiv:2507.05995 (2025).
- [21] L. Senthilkumar, T. Menzies, Can large language models improve se active learning via warm-starts? (2024).
- [22] T. Chen, M. Li, Adapting multi-objectivized software configuration tuning, Proc. of the ACM on Softw. Eng. (FSE) (2024).
- [23] T. Chen, M. Li, The weights can be harmful: Pareto search versus weighted search in multi-objective search-based software engineering, ACM Trans. Softw. Eng. and Methodology (2023).
- [24] V. Nair, A. Agrawal, J. Chen, W. Fu, G. Mathew, T. Menzies, L. Minku, M. Wagner, Z. Yu, Data-driven search-based software engineering, in: Proc. of the 15th Int. Conf. on Mining Software Repositories, 2018.
- [25] A. Lustosa, T. Menzies, Less noise, more signal: Drr for better optimizations of se tasks, arxiv:2503.21086 (2025).
- [26] B. W. Boehm, Software engineering economics, in: Software pioneers: Contributions to Softw. Eng., Springer, 2011.
- [27] A. H. Mohammadkhani, N. S. Bommi, M. Daboussi, O. Sabnis, C. Tantithamthavorn, H. Hemmati, A systematic literature review of explainable ai for software engineering, arxiv:2302.06065 (2023).
- [28] L. Arora, S. S. Girija, S. Kapoor, A. Raj, D. Pradhan, A. Shetgaonkar, Explainable artificial intelligence techniques for software development lifecycle: A phase-specific survey, arxiv:2505.07058 (2025).
- [29] C. Tantithamthavorn, J. Cito, H. Hemmati, S. Chandra, Explainable ai for se: Challenges and future directions, IEEE Softw. 40 (2023).
- [30] C. K. Tantithamthavorn, J. Jiarapakdee, Explainable ai for software engineering, in: Proc. ASE 2021, 2021.
- [31] R. Dwivedi, D. Dave, H. Naik, S. Singhal, R. Omer, P. Patel, B. Qian, Z. Wen, T. Shah, G. Morgan, R. Ranjan, Explainable ai (xai): Core ideas, techniques, and solutions, ACM Comput. Surv. (2023).
- [32] P. Biecek, T. Burzykowski, Explanatory model analysis: explore, explain, and examine predictive models, Chapman and Hall/CRC, 2021.

- [33] M. T. Ribeiro, S. Singh, C. Guestrin, " why should i trust you?" explaining the predictions of any classifier, in: Proc. KDD 2016, 2016.
- [34] S. M. Lundberg, S.-I. Lee, A unified approach to interpreting model predictions, Adv. Neural Inf. Process. Syst. (2017).
- [35] M. Staniak, P. Biecek, Explanations of model predictions with live and breakdown packages, The R Journal (2018).
- [36] D. Chen, W. Fu, R. Krishna, T. Menzies, Applications of psychological science for actionable analytics, in: Proc. FSE, 2018.
- [37] G. A. Miller, The magical number seven, plus or minus two: Some limits on our capacity for processing information., Psychol. Rev. (1956).
- [38] R. R. Hoffman, S. T. Mueller, G. Klein, M. Jalaeian, C. Tate, Explainable ai: roles and stakeholders, desirements and challenges, Frontiers in Computer Science (2023).
- [39] B. Goodman, S. Flaxman, European union regulations on algorithmic decision making and a "right to explanation", AI Mag. (2017).
- [40] A. Barredo Arrieta, N. Díaz-Rodríguez, J. Del Ser, A. Bennetot, S. Tabik, A. Barbado, S. Garcia, S. Gil-Lopez, D. Molina, R. Benjamins, R. Chatila, F. Herrera, Explainable artificial intelligence (xai): Concepts, taxonomies, opportunities and challenges toward responsible ai, Information Fusion (2020).
- [41] T. Menzies, Shockingly simple:" keys" for better ai for se, IEEE Software 38 (2) (2021) 114–118.
- [42] A. Agrawal, X. Yang, R. Agrawal, R. Yedida, X. Shen, T. Menzies, Simpler hyperparameter optimization for software analytics: Why, how, when?, IEEE Transactions on Software Engineering 48 (8) (2021) 2939–2954.
- [43] N. Awad, N. Mallik, F. Hutter, Dehb: Evolutionary hyperband for scalable, robust and efficient hyperparameter optimization, arXiv preprint arXiv:2105.09821 (2021).
- [44] S. Watanabe, Tree-structured parzen estimator: Understanding its algorithm components and their roles for better empirical performance, arxiv:2304.11127 (2023).
- [45] T. Menzies, A. Lustosa, Streamlining software reviews: Efficient predictive modeling with minimal examples, arXiv preprint arXiv:2405.12920 (2024).
- [46] C. A. Hoare, Algorithm 65: find, Communications of the ACM (1961).
- [47] J. Pearl, D. Mackenzie, The book of why: the new science of cause and effect, Basic books, 2018.
- [48] L. Breiman, Random forests, Machine learning (2001).
- [49] F. Hutter, H. H. Hoos, K. Leyton-Brown, Sequential model-based optimization for general algorithm configuration, in: Proceedings of the 5th International Conference on Learning and Intelligent Optimization, 2011.

- [50] M. Lindauer, K. Eggensperger, M. Feurer, A. Biedenkapp, D. Deng, C. Benjamins, T. Ruhkopf, R. Sass, F. Hutter, Smac3: a versatile bayesian optimization package for hyperparameter optimization, *J. Mach. Learn. Res.* (2022).
- [51] J. Bergstra, B. Komer, C. Eliasmith, D. Yamins, D. Cox, Hyperopt: A python library for model selection and hyperparameter optimization, *Computational Science & Discovery* (2015).
- [52] O. Arreche, T. Guntur, M. Abdallah, Xai-based feature selection for improved network intrusion detection systems, *arxiv:2410.10050* (2024).
- [53] A. Hinterleitner, T. Bartz-Beielstein, R. Schulz, S. Spengler, T. Winter, C. Leitenmeier, Enhancing feature selection and interpretability in ai regression tasks through feature attribution, *arxiv:2409.16787* (2024).
- [54] W. E. Marcilio, D. M. Eler, From explanations to feature selection: assessing shap values as feature selection mechanism, in: *Proc. SIBGRAPI 2020*, 2020.
- [55] M. Robnik-Šikonja, I. Kononenko, et al., An adaptation of relief for attribute estimation in regression, in: *Machine learning: Proceedings of the fourteenth international conference (ICML'97)*, 1997.
- [56] L. St, S. Wold, et al., Analysis of variance (anova), *Chemometrics and intelligent laboratory systems* (1989).
- [57] A. J. Scott, M. Knott, A cluster analysis method for grouping means in the analysis of variance, *Biometrics* (1974).
- [58] K. K. Ganguly, T. Menzies, Bingo! simple optimizers win big if problems collapse to a few buckets (2025).
- [59] T. Menzies, T. Chen, Moot repository of multi-objective optimization tests (2025).
URL <http://github.com/timm/moot>
- [60] R. G. Hamlet, Probable correctness theory, *Information processing letters* (1987).
- [61] J. Cohen, A power primer. (2016).
- [62] A. Lustosa, T. Menzies, isneak: Partial ordering as heuristics for model-based reasoning in software engineering, *IEEE Access* (2024).
- [63] S. Amershi, A. Begel, C. Bird, R. DeLine, H. Gall, E. Kamar, N. Nagappan, B. Nushi, T. Zimmermann, Software engineering for machine learning: A case study, in: *Proc. ICSE-SEIP 2019*, 2019.
- [64] F. J. Alcaide, J. R. Romero, A. Ramírez, Can explainable artificial intelligence support software modelers in model comprehension?, *Software and Syst. Modeling* (2025).
- [65] R. R. Hoffman, S. T. Mueller, G. Klein, J. Litman, Metrics for explainable ai: Challenges and prospects, *arxiv:1812.04608* (2018).
- [66] D. N. Palacio, A. Velasco, N. Cooper, A. Rodriguez, K. Moran, D. Poshyvanyk,

- Toward a theory of causation for interpreting neural code models, IEEE Trans. Softw. Eng. (2024).
- [67] I. Giagkiozis, P. Fleming, Methods for multi-objective optimization: An analysis, Inf. Sci. (2015).